

From Zero to nsF5

Steganography

Information Hiding x Machine Learning x Engineering

A beginner-friendly handbook for the nsF5 image steganography project: start with pixels and bits, learn LSB hiding, chi-square and RS steganalysis, Hamming matrix embedding, F5/nsF5 and wet paper coding, SHA-256 keying, and supervised machine-learning detection - then finish with C++/GPU engineering and your own capstone experiments.

Suggested pace: at least 12 weeks (quick-start); reserve 6-12 months to go deep x 6-8 hours per week

For undergraduates new to Python, security, and machine learning

Method: read the intuition -> open the project -> run experiments

Introduction - How to Use This Handbook

This is a handbook that teaches through code. It does not replace a textbook. Instead, it arranges the information-hiding and machine-learning ideas behind F:\Steganography in an order a beginner can actually follow: intuition and examples first, then the project implementation, then hands-on experiments.

0.1 Who This Is For

Assumed starting point - the handbook fills every other gap for you:

- Comfort with a Windows PC and installing software;
- Some college math exposure, or willingness to skip a formula and return later;
- A little programming is helpful but not required - Chapter 2 brings Python up to working level;
- At least 12 weeks of 6-8 hours per week; for a deep 6-12 month track see Appendix F.

Tip | If you have never installed Python, do not worry: the first two weeks are designed exactly for that situation. The goal is not to become a programmer but to become someone who can read and modify this project.

0.2 Reading Conventions

Marker	Meaning
Tip	Plain-language explanation or analogy that removes a common stumbling block
Watch out	A trap beginners frequently hit: types, mismatched parameters, misread statistics

Marker	Meaning
Try it	A hands-on experiment you must run; reading alone does not count as learning
Think about it	An open question with no single right answer, used to test real understanding
Read the code	Open a specific file and map the concept onto the implementation

0.3 The 12-Week Quick-Start Roadmap (deep 6-12 month track in Appendix F)

The route follows a natural order: carrier basics -> steganography -> steganalysis -> machine learning -> engineering -> capstone. Every week maps to runnable code or a reproducible experiment:

Week	Topic	Chapter	Hands-on outcome
1	Digital images and binary	Ch. 1	Inspect pixels and bit planes yourself
1 (supplement)	JPEG and DCT coefficients	Ch. 1½	Explain how the quantization table governs capacity
2	Python / NumPy / Pillow	Ch. 2	Run the GUI or run_e2e.py; read/write image arrays
3-4	LSB hiding and blind steganalysis	Ch. 3	Hide a message, then detect it with chi-square/RS
5	Matrix embedding and F5	Ch. 4	Use Hamming codes to change less and hide more
6	nsF5, wet paper, hash keying	Ch. 5-6	Explain the wet-paper solver in ns5_core.py
7	Machine-learning foundations	Ch. 7	Explain features, labels, overfitting, AUC

Week	Topic	Chapter	Hands-on outcome
8-9	ML-based steganalysis	Ch. 8	Run the v1 and v2 pipelines; explain SRM and dual models
10	C++ / GPU / data engineering	Ch. 9	Understand acceleration, self-checks, multi-source data
11-12	Capstone and presentation	Ch. 10-11	Complete and present a small improvement experiment

Try it | Copy this table into a weekly checklist (Appendix C has a ready-made one). Tick a row every weekend and answer that chapter's "Think about it" question.

0.4 What You Will Be Able to Do

- Explain steganography, steganalysis, embedding efficiency, shrinkage, wet paper coding, and syndromes in your own words;
- Say why naive LSB hiding is exposed by chi-square and RS analysis;
- Work one $p=3$ Hamming embedding example by hand (at most one coefficient changed per block);
- Explain the real difference between F5 and nsF5, and why eliminating shrinkage matters;
- Explain how image-hash keying gives decoder self-synchronization, and where its limits are;
- Explain the full supervised-learning pipeline, including why photo-grouped cross-validation is essential;
- Understand the v1 11-D features and the v2 143-D features, including SRM statistics and the 143d/53d dual-model strategy;

- Explain what C++ DLLs and GPU code accelerate and why bit-level consistency checks matter;
- Complete a small, honest improvement experiment and write up the results.

0.5 Project Map: Know the Code Before You Study It

The project is a research tool: it runs algorithm experiments and ships as a GUI application. Remember these files first; every chapter returns to them:

File	Role	Chapter
src/image_io.py	Image I/O wrappers for 8-bit grayscale and color images	Ch. 2
src/ns5_core.py	Algorithm core: Hamming codes, wet paper, hash keying, embed/extract	Ch. 3-6
src/steganalysis.py	Blind steganalysis: chi-square, RS, combined verdict	Ch. 3
src/efficiency.py	Theoretical and measured code-family / efficiency curves	Ch. 4
src/matrix_demo.py	Teaching demo of syndrome lookup	Ch. 4
src/fsfeatures.py + cpp/fsfeatures.dll	ctypes binding for the v1 11-D features	Ch. 8
src/featurize_v2.py + src/srm_filter.py	v1.4: SRM preprocessing and the 143-D v2 feature set	Ch. 8
src/make_dataset.py / train_model.py	Dataset generation (v1/v2, SRM, multi-source) and training	Ch. 8
src/ml_predict.py	Single-image ML prediction (143d/53d dual models + sensitivity)	Ch. 8
src/cppembed.py + cpp/nsf5embed.dll	C++ embed/shuffle hot path and self-checks	Ch. 9
gpu/*.py	PyTorch batched features (v1 11-D / v2 143-D) and	Ch. 9

File	Role	Chapter
	GPU training	
src/gui.py	Tkinter GUI with teaching panels	Ch. 9
src/test_core.py and friends	Regression tests for embedding, analysis, false positives	All

Read the code | Keep the project window open from now on. Whenever the handbook names a file, switch to it. From Chapter 2 onward most topics assume you run before you read.

0.6 Learning Tips

- **Run first, understand later.** End-to-end scripts remove most paper confusion;
- **Digest formulas twice.** First pass: conclusion and intuition. Second pass: derivation;
- **Retell in your own words.** If you can explain a chapter summary to a friend, you own it;
- **Keep an experiment log.** Parameters and outputs are the most valuable material for your capstone;
- **Trust the tests.** test_core.py and test_steg.py are the fastest judge of whether you broke something.

Think about it | Before you start, spend five minutes answering: what is the difference between encryption and steganography? If you cannot say, that is perfect - Chapter 3 starts there.

0.7 What Changed for v1.4.0 (2026-09-06)

The first draft of this handbook tracked project v1.3. On September 6 the project moved to v1.4.0, and this edition is synchronized with it. Chapters 1-6 (the steganography

algorithms) are unchanged; the machine-learning and engineering chapters now include sections 8.8 and 9.7, and older scores are labeled as "v1 baselines".

- ML detection grew from 11-D features with LR/XGB (AUC ~0.75-0.79) to a 143-D LightGBM default (8-split mean 0.8980) plus a 53-D interpretable model (0.8461): the 143-D model is the more accurate one, the 53-D one explains every dimension. See 8.8 and docs/RESULTS.md;
- New SRM high-pass preprocessing, 143-D v2 features, 12 embedding variants, real JPEG clean samples, and a full BOSSbase multi-source dataset; see 8.8 and 9.7;
- The repository moved to a new GitHub home (Yukinoshita-lin/nsf5-steganography) and the license is now Apache-2.0 with a NOTICE file;
- The glossary, command sheet, weekly checklist, and concept-to-code map were extended for v1.4.

Chapter 1 - Digital Images and Binary (Week 1)

Try it | Goals: cement the intuition that an image is a matrix of numbers; do binary and byte arithmetic by hand; explain why both the eye and statistics tolerate small pixel perturbations.

1.1 A Digital Image Is Just a Grid of Numbers

Zoom into an **8-bit grayscale image** and you will find a two-dimensional table of integers from 0 to 255: 0 is black, 255 is white, and the values between them are shades of gray. The computer stores the dimensions and the numbers - not the picture your eye sees.

A **color image** is three such tables stacked together: R (red), G (green), and B (blue). Each pixel holds three 0-255 values; an orange pixel might be R=255, G=128, B=0.

In code, a grayscale image is a 2-D array and a color image is a 3-D array. The project wraps this in `src/image_io.py: load_as_gray()` reads any supported image as a grayscale array, and `save_image()` writes an array back to disk.

Tip | Mathematics says rows and columns; image processing says height and width. A 512x512 image has 512 rows and 512 columns, so the array shape is (512, 512) - rows first.

1.2 Binary, Bytes, and Bits

A computer only understands 0 and 1. One 0/1 value is **1 bit**; eight bits form **1 byte**, which can represent $2^8 = 256$ combinations - exactly the grayscale range 0-255.

For example, decimal $200 = 128 + 64 + 8$, which is binary `11001000`. Its **least significant bit (LSB)** is the rightmost 0. Flipping 200 to 201 changes only the LSB, and the human eye cannot tell grayscale 200 from 201.

The LSB is the single most important concept in this handbook: LSB steganography is simply hiding secret bits inside the LSBs of pixels.

Try it | Open a Python prompt (or a scratch script) and verify this for yourself:

Experience: decimal, binary, and the LSB

```
v = 200
print(bin(v))           # 0b11001000
print(v & 1)            # lowest bit = 0
print(v ^ 1)           # flip lowest bit = 201
print(v & 0b11111110)  # clear lowest bit = 200
print(v & 0b11111110 | 1) # clear then set = 201
```

1.3 Why Images Have Room to Hide Things

Images are full of **redundancy**, in two senses:

- **Visual redundancy:** the eye is insensitive to tiny brightness changes and fine detail; changing one pixel from 128 to 129 is nearly invisible;
- **Statistical redundancy:** neighboring pixels in natural images are highly correlated; formats like JPEG remove this redundancy to shrink files;
- **The LSB plane looks like noise:** the lowest bits of a natural photo already look random, which gives a natural camouflage environment.

Compression and steganography therefore fight over the same resource: compression wants to remove redundancy; steganography wants to use it as hiding space. This is also why many hiding algorithms are designed for a specific file format.

Watch out | Do not confuse "invisible" with "safe." Invisibility is only the weakest requirement. A real analyst uses statistics, and naive LSB replacement leaves very clear statistical traces - exactly what chi-square and RS analysis catch in Chapter 3.

1.4 Bit Planes: Split an Image into 8 Layers

Write each grayscale value as 8 bits, then group pixels by bit position. You get eight binary images called **bit planes**. The most significant plane (MSB) carries the coarse structure; the LSB plane looks like scattered noise.

The intuition follows immediately: since the LSB plane is already near-random, inserting pseudo-random secret bits is hard to spot by eye or by simple histograms. But that same randomization changes internal structure, and smarter statistical tools can measure it.

Counting pixels and LSB values (numpy operations used everywhere in the project)

```
import numpy as np
img = np.asarray([[200, 201], [128, 129]], dtype=np.uint8)
print(img.shape, img.dtype)  # (2, 2) uint8
print(img & 1)               # LSB plane [[0,1],[0,1]]
print(np.unpackbits(np.array([200], dtype=np.uint8)))
# [1 1 0 0 1 0 0 0]  <- MSB first
```

1.5 Back to the Project: Meet cover and stego

Steganography uses two fixed names: the original carrier is the **cover**; the image after hiding is the **stego**. The project uses them everywhere: `img/cover.png` is the demo carrier and `output/stego_*.png` files are embedded results.

Read the code | Open step 1 of `src/run_e2e.py`: numpy builds a smooth 256x256 demo image with `np.linspace` gradients plus small noise and saves it as `img/cover.png`. Understanding that snippet means understanding how a natural-looking image grows out of numbers.

1.6 Summary and Self-Check

- A grayscale image is a 0-255 integer matrix; a color image has three channel matrices;

- The LSB is the lowest bit of a pixel value, and flipping it is visually negligible;
- Compression removes redundancy while steganography exploits it - they are opposing forces;
- cover = original carrier; stego = image after embedding.

Think about it | Is a pure black image (all zeros) a good cover for LSB hiding? What about a pure-noise image? Write your guesses down and return after Chapter 3.

Chapter 1½ - What is JPEG? What is a DCT coefficient? (The compressed-domain battlefield)

Tip |

This chapter was added in v1.9.0 (2026-10) in response to the feedback that the learning path should explain JPEG and DCT *before* LSB and steganography. It is numbered "1½" rather than renumbering every later chapter: the handbook has many "see chapter N" cross-references, ten per-chapter notebooks, and a fact-checking script — renumbering would cost far more than it gains. Read it after chapter 1 and before chapter 3.

Chapter 1 split an image into pixels and bit planes. The natural next step is LSB steganography. But nsF5 has two homes in this project: the pixel-domain implementation in `ns5_core.py` (main repo, modifies grayscale pixels), and — since v1.9.0 — the JPEG compressed-domain implementation (modifies quantized DCT coefficients, from the sister project `yccstego`, now bridged into the mainline). Textbook F5/nsF5 is the latter. Without understanding JPEG and DCT coefficients, the "+3→+2" in chapters 4 and 5 stays a slogan.

1.5.1 What is JPEG: a "lossy compressor"

JPEG is not just a file format; it is a **lossy compression pipeline** applied to every 8×8 pixel block:

1. **DCT transform:** rewrite the 8×8 pixel values as 64 "pattern weights" (next section);

2. **Quantization**: divide each weight by a step size from the quantization table and round — **this is where loss happens**: the bottom-right (high-frequency) steps are large, so rounding zeroes out most of them;

3. **Entropy coding**: serialize the surviving non-zero coefficients in zig-zag order and write the .jpg file.

Step 3 is reversible (you can decode it bit-exactly); step 2 is not (rounded information is gone). **All of compressed-domain steganography lives inside the product of step 2 — the quantized coefficients.**

Watch out |

This explains the chapter-3 FAQ line "JPEG destroys pixel LSBs": saving an LSB-stego PNG as JPEG re-runs the whole pipeline and quantization scrambles the LSB structure. Conversely, compressed-domain steganography writes the quantized coefficients themselves, and the encoder carries them faithfully through entropy coding — so the stego image **must remain a JPEG** and must **never be re-saved by another tool** (re-saving re-quantizes and destroys the payload). This is why the GUI saves JPEG-domain stego images as raw bytes.

1.5.2 What is a DCT coefficient: the "recipe" of an 8×8 block

Think of an 8×8 block as a tiny picture. The DCT (discrete cosine transform) provides 64 fixed "patterns": the top-left pattern is a uniform brightness (lowest frequency), and patterns get finer toward the bottom-right (high frequency). The DCT answers one question: **in what proportions must these 64 patterns be mixed to produce this block?** Those 64 proportions are the DCT coefficients.

- The first (top-left) coefficient is the **DC** term: the block's average brightness. It never carries the message;

- The other 63 are **AC** coefficients;
- Natural images concentrate energy in the top-left, so quantization zeroes out most bottom-right coefficients — **zeroed cells are not carriers; the surviving non-zero AC coefficients are.**

Try it |

The snippet below uses the very same DCT matrix and quantization table as the project's compressed-domain implementation (`yccstego/dct.py`) to quantize one 8×8 block at qualities 60, 85 and 95, counting non-zero AC carriers and $|c|=1$ "wet" points. Install the dependency first: `pip install yccstego`.

```
import numpy as np
from yccstego import dct as jdct

# Build an 8x8 block: smooth gradient (low frequency) + light noise (high)
rng = np.random.default_rng(3)
block = np.linspace(90, 200, 8)[None, :] + np.linspace(0, 60, 8)[:, None]
block = np.clip(block + rng.integers(-8, 9, (8, 8)), 0, 255).astype(float)

counts = {}
for q in (60, 85, 95):
    T = jdct.scale_qtable(jdct.LUM_QT, q)      # libjpeg-style quality scaling
    F = jdct.dct_blocks(block - 128)          # level shift, then DCT (JPEG way)
    Q = np.round(F / T).astype(int)           # quantization: loss happens here
    ac = Q.reshape(-1)[1:]                    # drop DC, 63 AC coefficients left
    nz = int(np.sum(ac != 0))
    wet = int(np.sum(np.abs(ac) == 1))
    counts[q] = nz
    print(f"quality {q}: {nz} non-zero AC carriers, {wet} wet ( $|c|=1$ ) points")
assert counts[95] > counts[60], "higher quality must leave more carriers"
print(f"quality 95 leaves more non-zero AC carriers than quality 60: {counts[95]} > {counts[60]}")
```

Observe |

Two patterns: raising quality from 60 to 95 visibly increases the carriers (smaller steps let more coefficients survive), and there are plenty of $|c|=1$ coefficients — which are

about to cause trouble.

1.5.3 nsF5 on coefficients: carriers, wet points, decrements

In JPEG-domain nsF5 the carriers are the **non-zero AC coefficients of quantized luma (Y) blocks**, each contributing one bit: $x = c \ \& \ 1$ (parity). Writing a bit is not a flip but **F5's decrement: subtract 1 from the magnitude, keep the sign**.

Coefficient	After	Note
+3	+2	magnitude -1, parity flips
-5	-4	sign preserved, LSB flipped
+1	wet, untouched	decrementing would hit 0 (shrinkage): it would vanish from the carrier set
-1	wet, untouched	same

Why shrinkage is a disaster: the extractor collects carriers in the same order; one coefficient shrinking from +1 to 0 removes it from the set and misaligns every later bit. That is F5's classic flaw, and exactly what nsF5 fixes with **wet-paper coding** (mark $|c|=1$ coefficients wet, solve on dry points) — chapter 5.

Try it |

Three lines to see the decrement/wet boundary:

```
for c in (3, -5, 1, -1):
    action = c - (1 if c > 0 else -1) if abs(c) > 1 else "wet, untouched"
    print(c, "->", action)
```

Think about it |

1. +3 and +2 share the same binary LSB of 1 — how does decrementing flip the bit at all? Hint: think about parity near zero, and why $|c|=1$ must not be decremented.
2. Compared with flipping (jumping between +3 and +4), decrementing piles magnitude around $|c|=1$. Is that pile-up the price of stealth, or its loophole? Chapter 8's "magnitude-1 fingerprint" turns it into a detector.

Watch out |

Keying works as in the pixel domain: the hiding path derives from "chroma-coefficient hash + password"; a mismatched p / password / quality makes extraction fail. The re-run contract is now identical in both domains: since yccstego 0.2.0 the wet-paper search seed derives from the input, so embedding is byte-deterministic; the experiment record stores the yccstego version used — re-runs on the same version get byte-level checks, and cross-version records fall back to "extraction matches" (older records stay compatible), as documented in `src/experiment.py`.

1.5.4 Run it on a real JPEG

The two snippets above looked at one block. Real embedding is done by yccstego: split the whole image, quantize block by block, run syndrome coding over all carriers, then entropy-code back to a standard .jpg. The bridge below drives exactly that (the GUI's JPEG domain and the CLI's `--jpeg` call the same code):

```

import sys
sys.path.insert(0, "src")          # needed for source runs; drop after pip install
import numpy as np
import jpegstego

rng = np.random.default_rng(7)
y = np.linspace(0, 220, 128)[None, :]
x = np.linspace(0, 90, 128)[:, None]
gray = np.clip(110 + y + x + rng.integers(-6, 7, (128, 128)), 0, 255).astype(np.uint8)
rgb = np.stack([gray, gray, gray], axis=-1)      # the JPEG domain eats RGB

jpg, rep = jpegstego.embed_jpeg(rgb, "Hello, DCT domain!", p=3, quality=85)
print("changed coefficients:", rep["carriers_changed"], "capacity(bit):",
      rep["capacity_bits"])

msg, _, tampered, head_match = jpegstego.extract_jpeg(jpg, p=3)
assert msg == "Hello, DCT domain!" and not tampered and head_match
print("round-trip matches:", msg)

```

Command-line equivalent (after installation):

```

nsf5stego embed cover.png --jpeg -m "secret" --quality 85
nsf5stego extract cover_stego.jpg --jpeg

```

1.5.5 Back to the code

Concept	File	Read first
Bridge (unified entry / degradation)	src/jpegstego.py	embed_jpeg / extract_jpeg
Block-level nsF5 (decrement + wet paper)	yccstego/yccstego/nsf5.py	_embed_block / solve_wet_paper
DCT and quantization tables	yccstego/yccstego/dct.py	_dct_matrix / scale_qtable / LUM_QT
JPEG-domain analysis ($ c =1$ fingerprint)	yccstego/yccstego/steganalysis.py	analyze_y

Concept	File	Read first
Re-run contract (two domains)	src/experiment.py	verify() and the module docstring
GUI domain switch	src/gui.py	_on_domain_change / _do_embed_jpeg

Think about it |

Embed the same 128×128 image once in the pixel domain (chapter 5's nsF5Pixel) and once in this chapter's JPEG domain, and count the writable positions: roughly 16,000 pixel bits versus a few thousand coefficients. What governs the gap? What happens at quality 60? This capacity intuition pays off in chapter 10's capstone.

Chapter 2 - Tooling: Python, NumPy, and Image I/O (Week 2)

Try it | Goals: install the environment, run the end-to-end script, and read/write image arrays with NumPy. From now on every experiment can be verified immediately in the project.

2.1 Setting Up

Dependencies are light: NumPy and Pillow for the core, plus scikit-learn/joblib/matplotlib for ML and plotting. Use Python 3.9+ (the README examples use 3.14; older versions work too).

Install in the project root and run the first self-tests

```
cd F:\Steganography
pip install numpy pillow
python src\test_core.py      # core algorithm self-test
python src\test_steg.py     # steganalysis self-test
python src\run_e2e.py       # embed -> decode -> analyze -> plot
```

Watch out | If the default `python` lacks `tkinter`, the GUI will not start; use an interpreter that ships `tkinter` (README suggests `C:\Python314\python.exe`). Command-line scripts work without it. On `ModuleNotFoundError`, first confirm you installed into the interpreter that runs the script: `python -m pip install ...` is the safe form.

2.2 Thinking in NumPy

Four NumPy concepts cover almost everything: **shape**, **dtype**, **slicing**, and **vectorized operations**. Steganography algorithms really just: take an array, rewrite the LSBs of some positions in some order, and store the array back.

Ten lines covering the four concepts

```
import numpy as np
a = np.arange(12).reshape(3, 4)      # 3 rows, 4 columns
print(a.shape, a.dtype)              # (3,4) int64
b = a[:, 1]                          # second column
c = a[0:2, 0:3]                      # top-left 2x3 block
x = np.zeros((64, 64), dtype=np.uint8)
x[10:20, 5:15] = 255                 # draw a white square
print((x & 1).sum())                  # count LSB=1 pixels
```

Note `dtype=np.uint8`: pixels are unsigned 8-bit integers 0-255. Arithmetic wraps around: `np.uint8(200) + 100` gives 44, not 300. The project flips LSBs with XOR 1 rather than +/- 1, precisely to avoid wrapping 0 below zero or 255 above the range.

2.3 Image I/O: Meet `image_io.py`

The core interface of `src/image_io.py`

```
from PIL import Image
import numpy as np

def load_as_gray(path):
    im = Image.open(path).convert("L")  # force grayscale
    return np.asarray(im, dtype=np.uint8)

def save_image(image, path):
    im = Image.fromarray(np.ascontiguousarray(image))
    im.save(path)
```

Read the code | Read the whole file yourself and compare `load_image()` with `load_as_gray()`, plus the supported extensions. Then run this snippet on the project's own demo image:

Try it: read an image, inspect its shape, save a copy

```
import sys; sys.path.insert(0, "src")
import image_io as IO
img = IO.load_as_gray("img/cover.png")
print(img.shape, img.dtype)          # (256,256) uint8
print(img.min(), img.max(), img.mean())
IO.save_image(img, "output/my_copy.png")
```

2.4 First End-to-End Run: run_e2e.py

Run `python src\run_e2e.py`. The script: generates a cover image -> embeds an English message with nsF5 $p=3$ (empty and non-empty password) -> decodes and compares -> runs blind analysis on clean and stego images -> plots the code-family/efficiency curves.

Try it | Read the console output carefully and copy three numbers: embedded bits, changed pixels, and the analysis probability for the clean versus stego image. By the end of Chapter 3 you will explain why they differ.

You do not need to understand embedding internals yet. This week's goal is only that the code runs on your machine and that the "change an LSB" intuition feels real.

2.5 Common Errors at a Glance

Error / symptom	Cause	Fix
ModuleNotFoundError	Installed into the wrong interpreter	<code>python -m pip install ...</code> with the interpreter you run
image too small to hold header + payload	Pixels below message capacity	Use a larger image, smaller p , or shorter message
UnicodeDecodeError / garbage	Default embedding supports UTF-8 (Chinese works out of the box)	Keep test messages ASCII; see Chapter 10 for UTF-8 extension ideas
GUI exits instantly / TclError	Interpreter has no tkinter	Run <code>src/gui.py</code> with a tkinter-enabled Python

Think about it | Why does `image_io` call `np.ascontiguousarray` before saving? Hint: slices and transposes can produce non-contiguous views, and many low-level libraries dislike them.

Chapter 3 - LSB Steganography and Its Statistical Fingerprint (Weeks 3-4)

Try it | Goals: complete the smallest possible "hide then detect" loop; understand what chi-square and RS analysis each measure; read the verdict logic in steganalysis.py.

3.1 Encryption vs. Steganography: The First Question

Dimension	Encryption	Steganography
Protects	Message content: unreadable without the key	The act of communication: nobody notices a secret exists
Visibility	Ciphertext clearly looks abnormal	The carrier looks completely normal
Typical use	Passwords, certificates, secure files	Covert channels, watermarking, forensic research
Relationship	Encrypt first, then hide (recommended)	Hiding conceals the existence of the secret

Tip | A useful mnemonic: **encryption protects the content; steganography protects the existence.** Professional systems use both: encrypt the message first, then hide the ciphertext. Ciphertext looks random, which actually makes it blend in better statistically.

3.2 A Minimal Runnable LSB Embedder

The most naive method is **LSB replacement**: write the secret bit stream into pixel LSBs one bit at a time. A grayscale image with $M \times N$ pixels holds up to $M \times N$ bits.

Textbook minimal LSB (for understanding only; the project uses matrix coding from Chapters 4-5)

```
import numpy as np
```

```
def text_to_bits(s):
    raw = s.encode("ascii")
    # 16-bit length header + 8 bits per character
    head = [(len(raw) >> i) & 1 for i in range(16)]
    body = np.unpackbits(np.frombuffer(raw, np.uint8))
    return np.concatenate([head, body]).astype(np.uint8)

def lsb_embed(cover, bits):
    flat = cover.ravel().copy()
    k = min(len(bits), flat.size)
    flat[:k] = (flat[:k] & 0xFE) | bits[:k] # clear LSB, write bit
    return flat.reshape(cover.shape)

def lsb_extract(stego, nbytes):
    flat = stego.ravel() & 1
    return bytes(np.packbits(flat[16:16 + nbytes*8]))
```

Watch out | Lossy formats are the enemy of LSB hiding. PNG and BMP store pixels losslessly, so the bit planes survive; JPEG recompression destroys LSBs. The project therefore saves PNG by default, and JPEG-domain hiding needs a separate design (the `yccstego` extension works on quantized DCT coefficients).

Why start every message with a 16-bit length header? The decoder must know how many bytes to read. `encode_string()` in `src/ns5_core.py` does exactly this: 16 little-endian length bits, then the payload bits. You just re-implemented that structure in the snippet above.

3.3 Why It Gets Caught: The Chi-Square Test

In natural images, adjacent grayscale levels $2i$ and $2i+1$ (say 100 and 101) usually occur with **unequal** frequencies. LSB replacement forces the pair counts toward equality: whenever a pixel moves from $2i$ to $2i+1$, or vice versa, the two histogram counts flatten. Westfeld's chi-square test measures exactly this:

1. Build the 256-bin grayscale histogram and pair levels (0,1), (2,3), ..., (254,255);

2. Under the "embedded" hypothesis each pair should be balanced, so the expected count is half the pair sum;
3. Compute the statistic $\sum(\text{observed} - \text{expected})^2 / \text{expected}$ over pairs with a positive sum;
4. Convert it to a p-value: a high p means "the data agree with uniform randomization" - suspicious;
5. The project repeats the statistic over 20 image prefixes and takes the median p for stability.

Watch the sign: here a **larger p is more suspicious**, because it says that fully randomized LSBs would easily produce what we observe - and a clean smooth image usually does not look like that.

Watch out | Natural noisy images fool the chi-square test. Digital photos already have near-random LSBs, so p is naturally high. The project adds a content-randomness correction: it estimates how noisy the image is from difference entropy before treating a high p as evidence. This is a key engineering detail for turning a heuristic into a usable tool.

3.4 RS Analysis: Structural Collapse of the LSB Plane

Fridrich's RS analysis takes a different view: instead of comparing pair counts, it examines the **spatial structure** of the LSB plane. Group the pixel stream into groups of four, define a smoothness discriminant f (sum of absolute differences inside the group), then flip selected LSBs with a positive mask $+M$ and a negative mask $-M$, and count groups where f grows (regular R) or shrinks (singular S).

Clean natural images show a strong regular tendency under the negative mask: $G_n = (R_n - S_n) / N$ is clearly positive (project values for clean smooth images are often 0.3-0.7).

LSB replacement randomizes the plane and collapses this structure: Gn drops sharply. The project also outputs Gr and an estimated embedding rate.

Core RS logic from steganalysis.py (pseudocode; it illustrates grouping, not a runnable script)

```
groups = flat[:m].reshape(-1, 4)          # groups of 4 pixels
fg = np.abs(np.diff(groups, axis=1)).sum(axis=1) # original f
pos = (groups ^ np.array([0,1,1,0])) & 0xFF    # +M flips
neg = (groups ^ np.array([1,0,0,1])) & 0xFF    # -M flips
Rm = (f(pos) > fg).sum(); Sm = (f(pos) < fg).sum()
Rn = (f(neg) > fg).sum(); Sn = (f(neg) < fg).sum()
Gn = (Rn - Sn) / n                        # clearly positive when clean, ~0 when random
```

3.5 The Project's Combined Verdict: analyze()

`analyze()` in `src/steganalysis.py` does not trust a single statistic. It combines four families of signals:

- Median chi-square p (were gray-level pairs flattened?);
- RS collapse (how far Gn fell below a content-adaptive baseline);
- Difference entropy and LSB difference entropy (is the randomness native to the image?);
- The sequence of prefix p-values (is randomization local or global?).

The three sensitivity modes (Strict / Balanced / Loose) move the possible/highly-likely thresholds and probability pull factors - in other words, they choose an operating point between false positives and missed detections.

3.6 Experiment: Hide an English Sentence, Then Catch It

Real project APIs: embed -> save -> analyze (safe to run step by step)

```
import sys; sys.path.insert(0, "src")
import numpy as np
import image_io as IO
from ns5_core import embed_string, extract_string
import steganalysis as SA
```

```

clean = IO.load_as_gray("img/cover.png")
stego, report, nbits = embed_string(
    clean, "Hello nsF5!", method="nsF5", p=3, password="")
IO.save_image(stego, "output/lab3_stego.png")
print("changed pixels:", report["cover_changed"])
print("decoded:", extract_string(stego, method="nsF5", p=3))
for name, im in (("clean", clean), ("stego", stego)):
    r = SA.analyze(im)
    print(name, "Gn=%.3f" % r["RS_Gn"],
          "chi2p=%.3f" % r["chi2_pvalue"],
          "prob=%.2f" % r["stego_probability"], r["verdict"])

```

Try it | Re-run with a 5000-character message (see `src/test_steg.py`) and watch changed-pixel share, Gn drop, and probability rise. Then try a genuinely noisy photo and see whether chi-square and RS can still separate clean from stego.

3.7 Summary and Self-Check

- LSB replacement flattens gray-level pair counts (chi-square) and destroys LSB-plane structure (RS);
- A high p is not safety; check whether the image is naturally random first;
- False positives and missed detections trade off; the sensitivity setting picks the operating point;
- The project's `analyze()` combines statistical signals with a content baseline.

Think about it | Naive LSB hiding embeds 1 bit per pixel and changes half of all pixels on average. If you hide very little - under 1% of pixels - can chi-square still catch it? RS? Keep these questions in mind for Chapter 4.

Chapter 4 - Matrix Embedding and F5: Change Less, Hide More (Week 5)

Try it | Goals: understand why embedding efficiency matters; work one $p=3$ Hamming syndrome-embedding example by hand; know where F5's shrinkage problem comes from.

4.1 From One Bit per Pixel to p Bits per Modification

Naive LSB costs about 0.5 pixel changes per bit. Suppose instead that every block of n pixels can carry p message bits while changing **at most one pixel**. Then the number of changes drops sharply. This idea is **matrix embedding**, and its mathematical engine is the binary Hamming code.

Embedding efficiency α = message bits carried per modification, with the theoretical value $\alpha(p) = p \cdot 2^p / (2^p - 1)$: about 2.67 for $p=2$, 3.43 for $p=3$, 4.27 for $p=4$. It grows with p , but so does the block length $n = 2^p - 1$.

Tip | Why is larger p "cheaper"? A Hamming code uses $n=2^p-1$ positions to carry p bits. Larger p means less overhead per position, so one modification carries more information. The cost is bigger blocks and stricter carrier requirements.

4.2 Hamming Code Crash Course: Parity-Check Matrix and Syndrome

Let block length $n = 2^p - 1$ and build a $p \times n$ **parity-check matrix H** whose columns are all nonzero vectors of $\text{GF}(2)^p$. For $p=3$, the seven columns are the binary representations of 1..7. Collect the LSBs of a block into a column vector x and define the syndrome $s = H \cdot x \pmod{2}$, a p -bit vector.

To embed p message bits m , we want to flip individual positions so the new syndrome equals m . Because every column of H is distinct, flipping position j changes the syndrome by $s \text{ XOR } H_j$ - so **any needed syndrome change corresponds to exactly one column**.

1. Compute the current syndrome $s = H * x$;
2. If $s == m$, this block needs no change;
3. Otherwise compute $d = s \text{ XOR } m$ (a nonzero p -bit vector);
4. Find the column j of H equal to d ;
5. Flip the LSB of position j ; now $H * x' = m$.

4.3 A Worked $p=3$ Example

Let the seven LSBs be $x = [0,1,0,1,1,0,1]$. With little-endian columns (LSB on top):
 $H_1=[1,0,0]$, $H_2=[0,1,0]$, $H_3=[1,1,0]$, $H_4=[0,0,1]$, and so on. The syndrome XORs the columns where x has 1 - positions 2, 4, 5, 7: $H_2 \text{ XOR } H_4 \text{ XOR } H_5 \text{ XOR } H_7 = [0,1,0] \text{ XOR } [0,0,1] \text{ XOR } [1,0,1] \text{ XOR } [1,1,1] = [0,0,1]$, so $s=[0,0,1]$.

Embed $m=[1,1,0]$: $d = s \text{ XOR } m = [0,0,1] \text{ XOR } [1,1,0] = [1,1,1]$. That matches H_7 , so flip the 7th LSB: $x'=[0,1,0,1,1,0,0]$. Check: $H * x' = [1,1,0] = m$. Done.

Seven pixels carry three bits, and on average only $(2^3-1)/2^3 \sim 87.5\%$ of blocks need a single flip - about 0.875 changes per block, or 3.43 bits per modification.

Try it | `src/matrix_demo.py` turns this into a clickable teaching panel (the "matrix coding demo" tab in the GUI). Call `demo_step(p, x, m_bin)` and inspect the returned s , d , and the matched column, then verify $H * x' == m$ yourself.

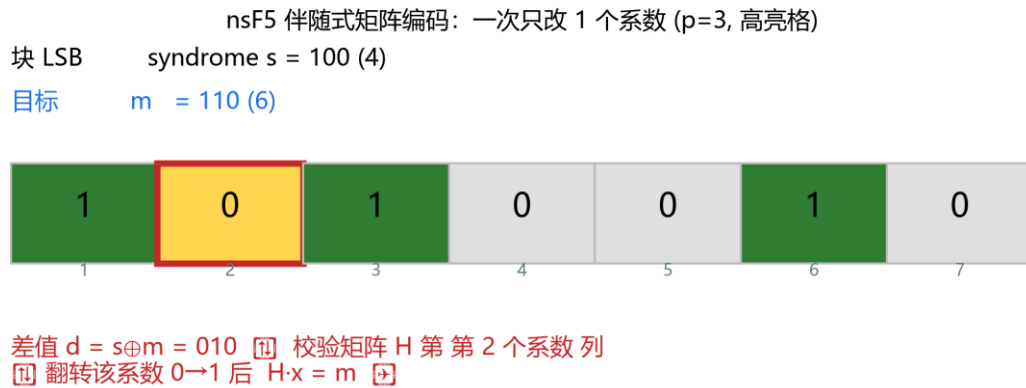


Fig. 4-1 Syndrome matrix embedding: at most one coefficient changed per block (project figure, $p=3$)

4.4 F5: Matrix Embedding on JPEG Coefficients

F5 (Westfeld, 2001) is JPEG steganography that applies matrix embedding to quantized DCT AC coefficients. Instead of flipping an LSB, it decreases the coefficient's **absolute value by 1** (e.g., $+3 \rightarrow +2$, $-5 \rightarrow -4$). That changes LSB parity without introducing large statistical anomalies.

The problem is **shrinkage**: when a coefficient with magnitude 1 is decreased, it becomes exactly 0. A zero coefficient means "no such frequency component" for JPEG, so F5 treats it as unusable, discards the block, and retries. Shrinkage lowers real payload, drops efficiency below the theoretical alpha, and leaves detectable statistical traces.

4.5 Back to the Project: MatrixEmbedding and Efficiency Curves

Read the code | Open the `MatrixEmbedding` class near line 216 of `src/ns5_core.py`. Its `_embed()` is the direct code version of section 4.3: compute s , compare to m , find the column, flip. `src/efficiency.py` plots theoretical $\alpha(p)$ against the measured efficiency.

MatrixEmbedding_embed() core (excerpt of the real code)

```
x1 = (c[pos] & 1).astype(np.uint8)    # take block LSBs
s = syndrome(H, x1)                    # s = H*x
if np.array_equal(s, m): continue       # already matched
d = (s ^ m).astype(np.uint8)           # difference
col = int(np.where(np.all(H == d[:, None], axis=0))[0][0])
c[pos[col]] ^= 1                       # flip the matched pixel
```

Generate the code-family and efficiency plot

```
import sys; sys.path.insert(0, "src")
from efficiency import plot_code_family_and_efficiency
path = plot_code_family_and_efficiency(p_max=6)
print(path) # output/efficiency.png
```

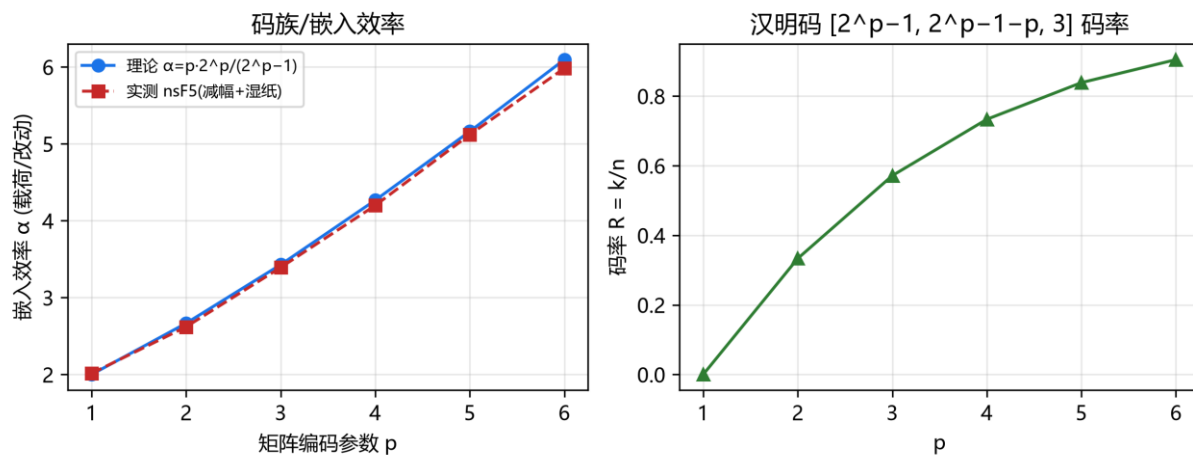


Fig. 4-2 Hamming code family and embedding efficiency: theory vs. measurement (project figure)

4.6 Summary and Self-Check

- Matrix embedding works on blocks: n positions carry p bits, at most one position changes;
- Distinct H columns mean every syndrome difference locates exactly one position to flip;
- Embedding efficiency $\alpha = p \cdot 2^p / (2^p - 1)$, rising with p but so does block length;

- F5 decreases JPEG coefficient magnitudes and "shrinks" when magnitude 1 hits zero.

Think about it | If F5 skips and retries a shrunk block, can the message still decode? How does the decoder know which blocks were skipped? If you cannot answer, that is fine - Chapter 5's nsF5 sidesteps the whole problem with wet paper coding.

Chapter 5 - nsF5 and Wet Paper Coding: The Project's Core Algorithm (Week 6)

Try it | Goals: explain why shrinkage damages efficiency; understand wet paper coding as "wet positions untouched, solve on dry positions"; walk through the complete nsF5 flow in `ns5_core.py`.

5.1 State the Problem Clearly: F5 Shrinkage

F5 modifies JPEG quantized coefficients by decreasing magnitude, which flips the coefficient's LSB parity each time. But a coefficient with magnitude 1 (e.g., +1 or -1) becomes zero when decreased. For JPEG, zero means the band is absent, so F5 abandons that block and looks for another one. That is **shrinkage**.

- Shrinkage lowers **actual capacity**: blocks are wasted and the message needs more positions;
- Shrinkage lowers **embedding efficiency** below $\alpha(p)$, because lost blocks must be compensated;
- Shrinkage leaves detectable traces: too many coefficients were driven to zero, which shows up in histograms.

The project makes a clever spatial-domain analogy: subtract 128 from each pixel to get a signed value $xv = \text{pixel} - 128$. Pixels with $|xv| \leq 1$ (that is, 127, 128, and 129) are the "danger zone" where decreasing the magnitude would land on or cross zero. The matrix-embedding shrinkage problem from JPEG becomes the same problem: those pixels cannot be ordinary modification targets.

5.2 Wet Paper Coding: Mark Wet Positions, Solve on Dry Ones

Wet paper coding, introduced by Fridrich and colleagues, answers one question: if some carrier positions cannot be touched (like ink on wet paper), can we still embed the full message using only the dry positions?

Yes - and the two parties do not need to agree in advance which positions are wet, as long as the algorithm can reconstruct the same wet/dry partition from the image content. The project does exactly this:

1. For each block, find positions whose magnitude decrease would hit zero/danger and mark them wet;
2. The real problem: find a modification vector e on the dry positions so the block syndrome becomes m ;
3. That is, solve $H[:, \text{dry}] * y = d$ over $GF(2)$, where $d = s \text{ XOR } m$;
4. The solver tries single dry-column hits, then pairs of dry columns, then a $GF(2)$ Gaussian-elimination fallback, keeping changes sparse;
5. The solution y is a 0/1 vector; positions with $y=1$ are the dry positions actually modified.

Tip | Why does pre-marking wet positions eliminate shrinkage? Shrinkage happened when a modification accidentally produced zero halfway through. nsF5 excludes all such positions in advance and solves only on safe dry positions, so every block embeds successfully: no retries, no surplus zeros.

5.3 Reading the Project: nsF5Pixel._embed()

In `src/ns5_core.py`, the `nsF5Pixel` class inherits from `MatrixEmbedding` and overrides only `_embed()`. Read that method line by line and you will see the complete nsF5 decision tree:

nsF5Pixel_embed() decision tree (pseudocode; logic matches the real code)

```
xv = c[pos].astype(np.int16) - 128    # pixel -> signed value
s = syndrome(H, (xv & 1).astype(np.uint8))
if s == m: continue                    # syndrome already matches
tc = matched_column(H, s ^ m)          # most direct candidate
if abs(xv[tc]) > 1:                    # safe to decrease
    c[pos[tc]] -= sign(xv[tc])         # one step toward 128
else:                                  # candidate is wet
    dry = [k for k in range(n) if abs(xv[k]) > 1]
    e = solve_wet_paper(H, dry, d)     # solve on dry positions
    if e: decrease every dry position selected by e
    else: fallback: flip the target LSB (keeps decodability)
```

5.4 The Three Solver Functions

Function	Purpose	Key point
solve_wet_paper(H, dry_cols, target)	Find a sparse solution on dry columns	Weight 1 -> weight 2 -> Gaussian fallback
gauss_solve_GF2(cols, b)	Solve a GF(2) linear system	Free variables set to 0 to minimize changes
permute_index(total, seed)	Deterministic pseudo-random permutation	splitmix64 + Fisher-Yates; C++ accelerated

Read the code | Open lines 145-215 of `src/ns5_core.py` and read the three functions against this table. Then run `test_wet_paper_needed()` in `src/test_core.py`: it forces pixels to 127/128/129 to create wet positions and still completes the embed/decode round trip.

5.5 Verification: Round Trips and Efficiency

Run the core self-test and compare the two methods

```
python src\test_core.py
# compare matrix vs nsF5 changed pixels:
import sys; sys.path.insert(0, "src")
import numpy as np; from ns5_core import embed_string
img = np.random.default_rng(0).integers(0, 256, (64, 64), np.uint8)
for method in ("matrix", "nsF5"):
```

```
stego, rep, nb = embed_string(img, "Hello nsF5!", method=method, p=3)
print(method, "changed", rep["cover_changed"], "pixels / capacity", nb)
```

On the same random image and message, matrix and nsF5 may change similar numbers of pixels, because danger-zone pixels (127/128/129) are rare in random data. The real difference appears when danger-zone pixels are dense: nsF5 still embeds without shrinkage while matrix embedding keeps failing.

5.6 Summary and Self-Check

- Shrinkage = a magnitude decrease hits zero, ruining the block, capacity, efficiency, and statistics;
- Wet paper coding marks untouchable positions as wet and solves the syndrome equation only on dry ones;
- nsF5 = matrix embedding + wet paper coding: no shrinkage, no retries;
- In the pixel domain the project uses $xv = \text{pixel} - 128$ and treats $|xv| \leq 1$ as wet.

Think about it | Why does wet paper coding not require the decoder to know the wet positions? Hint: the decoder only reads LSBs and computes syndromes; it never needs to know which positions the embedder skipped.

Chapter 6 - Passwords, SHA-256, and Keyed Security Design (Weeks 6-7)

Try it | Goals: understand why embedding positions must depend on a key; explain decoder self-synchronization and tamper perception; know the limits of this design.

6.1 What Happens When Positions Are Fixed?

Suppose embedding always starts at the top-left corner and moves in order. An attacker who knows the algorithm can: (1) read the LSBs of the first N pixels and recover the message; (2) copy the message to another "identical-looking" image (position-replacement attack); (3) perturb only the fixed region to destroy your message without touching the rest of the image.

Modern steganography therefore scrambles the embedding path with a **key-controlled pseudo-random order**. Without the password (or content hash), nobody knows which block holds the message and position-level attacks fail.

6.2 Two Seed Rules: Recognize the Image First, Then Find the Payload

The project splits pixels into a **header pool** (the first N_h blocks) and a **payload pool** (the remaining pixels). Two independent seeds are derived:

- **seed0 = derived from the password only**: scrambles the header pool. The header stores `cover_hash` - the first 16 bytes (128 bits) of the SHA-256 of the original image bytes;
- **seedB = derived from cover_hash + password**: scrambles the payload pool and block ordering.

To decode, the receiver rebuilds seed0 from the password, reads the header to recover cover_hash, and only then derives seedB to find the payload. A wrong password or a broken header kills the path immediately - that is what `test_pwd_mismatch()` verifies.

Seed derivation in the project (real code)

```
def derive_seed(image_bytes, password=""):
    base = hashlib.sha256(image_bytes).hexdigest()
    if password:
        base = hashlib.sha256(
            base.encode() + password.encode()).hexdigest()
    return int(base, 16)
```

6.3 Deterministic Shuffling: splitmix64 + Fisher-Yates

A seed alone is not enough; it must expand into a permutation of 1..N. The project uses splitmix64 as the pseudo-random source and performs Fisher-Yates: walk from the end to the front, each step swapping with a random earlier position. The same seed always gives the same permutation - the mathematical basis of self-synchronization.

Read the code | Open `permute_index()` in `src/ns5_core.py`: with the DLL present it calls C++ `nsf5_permute`; otherwise it runs an identical Python fallback. **Both implementations must produce exactly the same permutation**, otherwise an image embedded with the DLL cannot be decoded on a machine without it. `selfcheck()` in `src/cppembed.py` verifies cross-language consistency.

6.4 Tamper Perception: What It Can and Cannot Do

The embedder reports `cover_hash` (the original image hash). Any pixel change alters the SHA-256 of the image bytes. Recompute the hash of the received image and compare: mismatch means the image was touched. More elegantly, because the payload path depends on that hash, even a one-pixel change shifts the payload seed and the decode collapses or produces garbage.

Be honest about the boundary: this is **keying plus tamper perception**, not cryptographic **integrity authentication**. The header hash is not keyed with a MAC, so an attacker capable of re-embedding the whole image can update the header hash too. The README lists a keyed MAC as future work; keep the distinction between "detect ordinary tampering" and "resist a malicious re-embedder" clear.

6.5 Experiments: Wrong Password and One-Pixel Tampering

Experiment A: wrong password must fail; Experiment B: tamper one pixel

```
import sys; sys.path.insert(0, "src")
import numpy as np; import image_io as IO
from ns5_core import embed_string, extract_string, get_image_hash

img = IO.load_as_gray("img/cover.png")
stego, rep, _ = embed_string(img, "secret", method="nsF5",
                             p=3, password="A")
print(extract_string(stego, method="nsF5", p=3, password="B"))
tampered = stego.copy(); tampered[0, 0] ^= 1 # flip one pixel
print(get_image_hash(stego)[:16], "vs", get_image_hash(tampered)[:16])
print(extract_string(tampered, method="nsF5", p=3, password="A"))
```

Watch out | Experiment B may fail as a ValueError, garbage text, or a truncated message, depending on which block the pixel belongs to. Do not expect an exception every time: the engineering reality is "hash mismatch plus failed self-synchronization," not a clean error message.

6.6 Summary and Self-Check

- Fixed paths are predictable: readable, copyable, attackable; keyed paths make positions depend on password + content;
- The header stores a 128-bit content hash and the payload seed depends on it - decoding needs no external sync;

- Deterministic permutation (splitmix64 + Fisher-Yates) is the mathematical guarantee of self-synchronization;
- Keying is not integrity authentication; strong adversaries require a keyed MAC.

Think about it | If two different cover images hide the same message, will their stego images use the same hiding positions? Why or why not? (Hint: what does the payload seed depend on?)

Chapter 7 - Machine Learning Foundations (First Time Learners, Weeks 7-8)

Try it | Goals: build the full data -> feature -> model -> evaluation frame; explain overfitting, cross-validation, and data leakage; read ROC and AUC correctly.

7.1 Steganalysis Is a Binary Classification Problem

Change perspective: for any image, it is either a clean carrier (label 0) or a carrier that hid a message (label 1). Machine learning learns a function $f(\text{image}) \rightarrow 0/1$, or "stego probability." That is exactly what the project's supervised steganalysis does.

Supervised learning has four pieces: **data**, **features**, **model**, **evaluation**. Each concept below maps to project code immediately.

7.2 Data: Samples, Features, Labels

Version 1 compresses each image into 11 numbers called a **feature vector** and records its class (**label**). Version 1.4 adds a 143-D v2 feature set on top (Section 8.8). For teaching, start with the v1 table `data/dataset.csv`: 2,898 rows = 414 photos x (1 clean + 6 stego variants), of which 414 are clean and 2,484 are stego.

data/dataset.csv (real first two rows, truncated for display)

```
Rm,Sm,Rn,Sn,RS_Gr,RS_Gn,chi2_pvalue,diff_entropy,
lsb_diff_entropy,median_prefix_p,chi2_stat,label,photo_id,...
41316,4574,36608,5057,0.56,0.48,3.5e-278,1.99,0.986,2.7e-166,1719.3,0,0,...
40488,4907,35701,5342,0.54,0.46,2.0e-269,2.02,0.989,2.2e-155,1675.9,1,0,...
```

Notice `photo_id`: the clean version and all variants of one photo share an id. That unremarkable column is the key to preventing data leakage in 7.4 (v1.4's v2 datasets add more variants and real JPEG clean samples, but the grouping idea is unchanged).

7.3 How a Model Learns: Loss, Gradient, Logistic Regression

Start with the most explainable model - **logistic regression**: a linear combination $z = w \cdot x + b$ passes through the sigmoid $\sigma(z) = 1/(1 + e^{-z})$, producing a 0-1 probability.

Training means finding weights w that minimize the **loss** over all samples. Logistic regression typically uses cross-entropy: predicting 1 when the truth is 0 is punished, and the more confident the wrong prediction, the steeper the penalty. Optimizers such as gradient descent nudge w downhill until the loss stops falling.

Tip | You do not need to derive gradients now. Remember three sentences: **loss = how wrong the model is; training = find parameters that minimize loss; gradient descent = keep moving in the steepest downhill direction.** The project wraps all this in scikit-learn's `LogisticRegression`.

7.4 Overfitting, Cross-Validation, and Data Leakage

Performing well on data the model has already seen is easy. The real test is **unseen data** - that is generalization. A model that memorizes the training samples collapses on the test set: **overfitting**.

The standard defense is **cross-validation**: split the data into folds and rotate which fold is the validation set. But there is a subtle trap: all seven samples from one photo are highly similar. A random split can put a "sibling sample" of a test photo into training, which means the model has effectively peeked at the test photo - **data leakage** - and the AUC is unrealistically high.

Watch out | The project uses `GroupKFold`: splits are grouped by photo id, so every variant of a photo is either entirely in training or entirely in validation. That single choice in `cv_auc()` (`src/train_model.py`) decides whether the experiment is trustworthy. It is one of the most valuable machine-learning engineering lessons in this project.

7.5 Evaluation Metrics: Accuracy Is Not Enough

Stego detection has imbalanced classes and asymmetric costs (flagging a clean image differs from missing a stego image), so one number is not enough:

Metric	Question it answers	Intuition
Accuracy	Overall fraction correct	Misleading when classes are imbalanced
Precision	Of flagged stego, how many truly are	Low false positives -> high precision
Recall	Of true stego, how many were caught	Low misses -> high recall
F1	Harmonic mean of precision and recall	Compromise when you want both
ROC / AUC	Ranking ability independent of threshold	AUC 0.5 = random guessing

Models output probabilities, so a **threshold** must turn them into 0/1 decisions. Higher thresholds are conservative (fewer false positives, more misses); lower thresholds are aggressive. Training picks a **Youden** threshold where true-positive rate minus false-positive rate is largest, and also computes a separate low-false-positive threshold (FP <= 10%) for the Strict mode.

Youden threshold (real logic)

```
from sklearn.metrics import roc_curve
fpr, tpr, th = roc_curve(y_true, proba)
j = tpr - fpr                                # Youden J
best = float(th[np.argmax(j)])                # point farthest from random
```

Try it | After this section, run `python src\train_model.py` once. You do not need every line of output; find three: CV-AUC, held-out test AUC, and low-FP detection rate. Chapter 8 explains them line by line.

7.6 Summary and Self-Check

- Supervised learning = learn a function from features to answers using labeled samples;
- Random splits leak information between same-source samples; GroupKFold splits are the honest option;
- AUC measures ranking; the threshold decides the operating point;
- False-positive rate and detection rate trade off; Youden and low-FP thresholds are two choices.

Think about it | A paper reports steganalysis AUC 0.95 but never says how data were split. What do you suspect first? Write it down - that instinct is the start of critical ML evaluation.

Chapter 8 - Machine-Learning Steganalysis (Weeks 8-9)

Try it | Goals: understand why the project uses 11-D statistical features instead of raw-pixel CNNs; know what each feature measures; run the data -> train -> evaluate -> GUI pipeline; explain the v1.4 SRM / 143-D / dual-model story.

8.1 The Counterintuitive Question: Why Not a Raw-Pixel CNN?

Feeding raw pixels to a deep CNN does not work well here. The project tried multiple architectures on 414 photos, and validation AUC hovered around 0.50 - no generalization at all. Why?

- Steganographic changes are an extremely weak high-frequency signal (LSB layer), far below objects and textures;
- A CNN easily learns photo content rather than hiding traces, and fails as soon as test photos look different;
- The number of truly independent samples is the number of photos (hundreds), while deep models usually need thousands of independent sources.

The project therefore uses **domain knowledge as features plus simple classifiers**: hand-design features that directly measure LSB statistical footprints. On the same small dataset, the v1 11-D feature route reached AUC ~0.75-0.79, and every feature is explainable. v1.4 raised this to 0.90+ with 143-D features and LightGBM (Section 8.8), while the "features beat raw CNN" conclusion stands.

Tip | This is not a "deep learning is useless" claim. It is a lesson in modeling under sample constraints: first verify the signal with strong explainable features, then consider more complex models. Many engineering problems follow the same order.

8.2 Meet the v1 11-D Features

The v1 features are all computed by the C++ library `cpp/fsfeatures.dll` (Python binding in `src/fsfeatures.py`); the 143-D v2 set is assembled in `src/featurize_v2.py`. The 11 features fall into three groups:

Feature	Meaning	After embedding
Rm / Sm / Gr	Regular/singular group counts and gap under +M	Gr drops (structure damaged)
Rn / Sn / Gn	Counts and gap under -M	Gn collapses (core signal)
chi2_stat / chi2_pvalue	Chi-square statistic and p-value	Gray pairs balance, p rises
diff_entropy	Difference entropy: native texture roughness	Used to identify naturally noisy images
lsb_diff_entropy	LSB-plane difference entropy	Approaches 1 as LSBs randomize
median_prefix_p	Median chi-square p over 20 prefixes	Rises with global randomization

Read the code | Open `src/steganalysis.py` and find `diff_entropy()` and `lsb_diff_entropy()`: they estimate the randomness baseline with Shannon entropy of adjacent differences. Then open `src/fsfeatures.py` to see how the feature order is passed to C++.

8.3 Where Data Come From: The "1 + 6" Design of `make_dataset.py`

Training data cannot come from a few hand-hid messages. `src/make_dataset.py` produces one clean image plus six stego variants per source photo (v1.4 also supports 12 v2 variants; see 8.8):

Variant (method, p, density)	Strength	Purpose
clean	-	Provides clean negative samples

Variant (method, p, density)	Strength	Purpose
nsF5, p=3, d=0.25	weak	Close to realistic covert communication
nsF5, p=3, d=0.55	medium	Moderate strength
nsF5, p=2, d=0.35	weak-medium	Lower p is less efficient, leaves more traces
nsF5, p=2, d=0.85	strong	Heavy embedding, obvious footprint
matrix, p=3, d=0.50	medium	Compare a second method
matrix, p=2, d=0.80	strong	High-density plain matrix embedding
v1.4 adds +6	-	nsF5 p3 d0.40; matrix p3 d0.40/0.60; lsb d0.30/0.50/0.70

All images are resized to 512x512 grayscale; features come from C++ and stego variants are produced by the C++ embedding path, so training and inference stay consistent.

8.4 How to Read the Training Script: `train_model.py`

`python src\train_model.py` is one honest evaluation pipeline:

1. Load the table and drop rows with missing features;
2. Reserve 25% as a held-out test set with `GroupShuffleSplit` by `photo_id` (those photos never enter training);
3. Run 5-fold `GroupKFold` on the training pool, comparing `LogisticRegression` / `RandomForest` / `GradientBoosting` / `XGBoost` (v1.4 adds `LightGBM` tuning and stacking);
4. Pick the best CV model, carve 20% of the training pool as a calibration set, and choose Youden plus low-FP thresholds;
5. Retrain on the full training pool and report accuracy, balanced accuracy, AUC, and classification report on held-out data;

6. Report per-(method, p, density) detection rates;
7. Save model and thresholds (v1.4 keeps dual versions; see 8.8).

Watch out | The calibration set is for threshold selection and the test set is used exactly once. Re-tuning thresholds on the test set dirties it. Training/calibration/test is the same three-layer discipline industry uses.

8.5 Reading Results: ROC, AUC, and Per-Density Rates

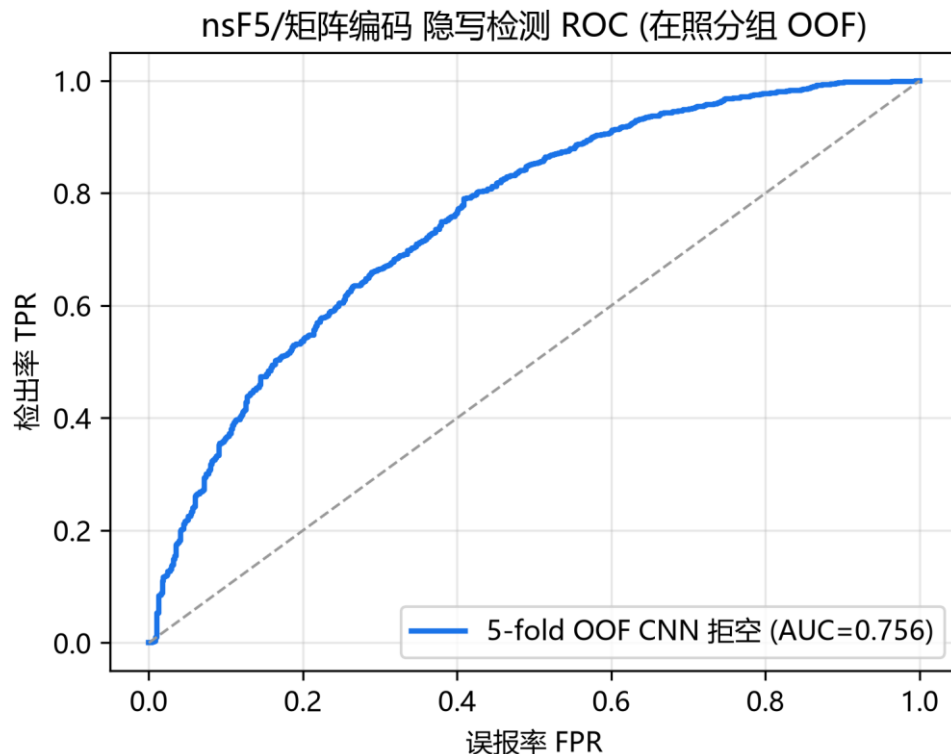


Fig. 8-1 Steganalysis ROC (photo-grouped held-out folds; v1 11-D baseline, AUC ~0.756)

The ROC curve sweeps the threshold from 1 to 0 and plots detection rate against false-positive rate. AUC is the area under it: 0.5 is random. The v1 baseline around 0.75 means "usually ranks stego above clean," but weak densities still escape. The v1.4 dual models

reach an 8-split mean AUC of 0.8980 (143-D) / 0.8461 (53-D); see docs/RESULTS.md for the protocol.

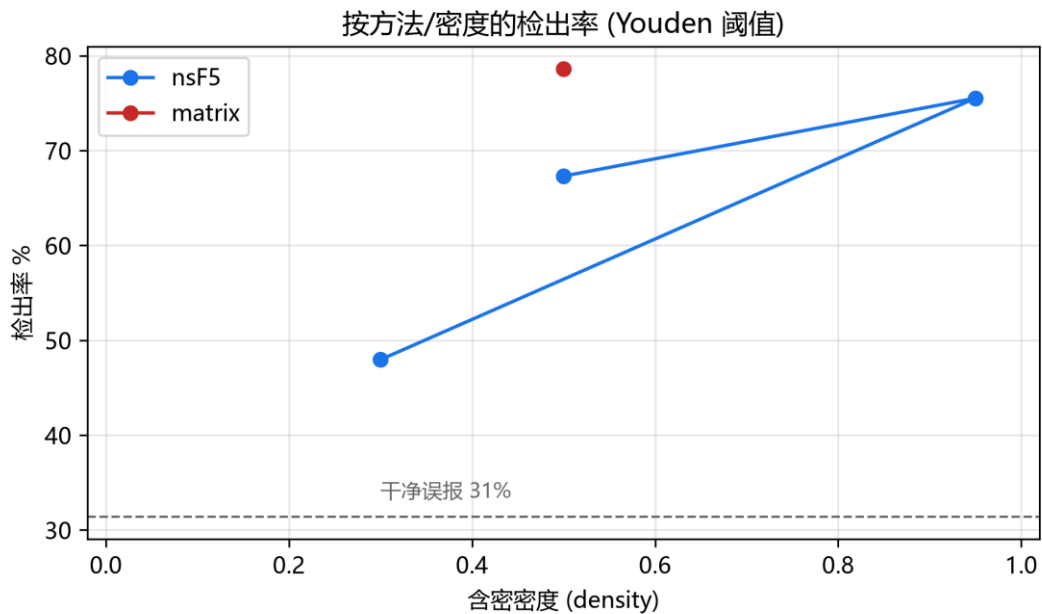


Fig. 8-2 Per-method/per-density detection (Youden threshold; v1 6-variant baseline): strong densities high, weak densities drop

Read the code | Compare experiments/data/density_grid.csv (real statistics produced by the chapter-5 experiments): LSB bands are near 100%, matrix p2 d0.80 is about 95%, while weak nsF5 p3 d0.25 reaches only ~47% and nsF5 p3 d0.55 ~55%. Weak embedding being harder to detect is not mysticism; it is the direct result of a smaller statistical footprint.

8.6 Sensitivity in the GUI and in ml_predict.py

MLPredictor in `src/ml_predict.py` preprocesses the image (grayscale -> 512x512), then automatically picks the v1 11-D or v2 143-D feature path based on the model's stored feature list, and outputs a stego probability. Sensitivity works by switching thresholds:

Sensitivity	ML threshold	Effect
-------------	--------------	--------

Sensitivity	ML threshold	Effect
Strict (low FP)	threshold_low_fp	Fewest clean-image errors; more weak stego missed
Balanced	Youden threshold	Balanced detection vs. false positives; default
Loose (high detection)	Youden x 0.75	Catches more weak densities; false positives rise

The GUI shows heuristic and ML probabilities side by side and asks you to **cross-check**, not trust one number - a deliberate honesty feature of the project.

8.7 Experiments: From Training to Single-Image Prediction

The full pipeline (minutes to tens of minutes, machine dependent)

```
# 1) generate a dataset (point it at your own photo folder)
python src\make_dataset.py data\campus_jpg --out campus

# 2) train and evaluate
python src\train_model.py

# 3) single-image ML prediction
import sys; sys.path.insert(0, "src")
import numpy as np
from PIL import Image
from ns5_core import embed_string
from ml_predict import get_predictor
a = np.asarray(Image.open("img/cover.png").convert("L"))
s, _, _ = embed_string(a, "ML test", method="nsF5", p=3)
print(get_predictor().predict(s))
```

Try it | Run an honesty experiment: predict on a clean photo (should be near clean), then embed a weak-band message (nsF5 p=3 near density 0.25) and predict again. The probability may move only slightly - that is the README warning about weak densities, live.

8.8 v1.4.0 Update: SRM, 143-D Features, and Dual Models (2026-09)

Sections 8.1-8.7 describe the v1.0-v1.3 route: 11-D features with LR/XGB and AUC around 0.75-0.79. v1.4.0 upgraded three things: SRM high-pass preprocessing, a 143-D

v2 feature set with 12 variants, and finally a dual LightGBM model strategy. The story below follows the project's actual order so that the README's long experiment tables make sense.

8.8.1 SRM High-Pass Filtering: Same-Source Gain, Cross-Source Loss

SRM (Spatial Rich Model) is a family of high-pass filters from the Fridrich framework. `src/srm_filter.py` implements the standard 30 kernels (8 first-order, 4 second-order, 8 third-order differences, 8 edge kernels, 2 square kernels), filters the image, takes the maximum absolute response per pixel, clips to ± 4 , and rescales to a single uint8 "enhanced image" that feeds the existing feature extractors. CPU (numpy) and GPU (torch) implementations are exposed through `make_dataset.py --preprocess srm` and the GPU `use_srm=True` flag.

Metric	Same-source baseline	Same-source + SRM
5-fold CV-AUC	0.7594	0.7922
Held-out test AUC	0.7811	0.8085
Weak nsF5 p3 d0.25 detection	51.9%	62.5%
nsF5 p2 d0.35 detection	76.0%	90.4%
Low-FP stego detection	41.2%	50.3%
Clean false positives (low-FP thr)	9.6%	9.6%

Watch out (v1-era comparison; not reproduced in the current revision) | At the time the gain appeared only within the same source domain. In cross-source training (campus + full BOSSbase), SRM pulls CPU test AUC from 0.741 down to 0.704 and GPU validation AUC from 0.651 down to 0.572. High-pass filtering also flattens the signals that distinguish different cameras/compression sources. That is why the default model is trained on merged data without SRM, while the SRM campus model is kept as a separate same-source artifact.

8.8.2 The v2 143-D Feature Set and 12 Variants

`src/featurize_v2.py` adds to the 11 base features: mean / absolute mean / standard deviation over 30 SRM residuals (90 D), 20 full-image prefix chi-square p-values, 20 LSB-prefix p-values, plus texture_noise and est_rate (2 D): $11 + 90 + 20 + 2 + 20 = 143$. The training variants grow from 6 to 12: nsF5 p3 d0.40, matrix p3 d0.40/0.60, and lsb d0.30/0.50/0.70 are added.

Strict A/B results (same test photo groups, 5-fold GroupKFold OOF): the 12-variant v2 dataset alone adds +0.019 (0.7868 vs 0.7678 at equal 11-D features); adding the 143-D set adds another +0.027 (OOF 0.8143, held-out 0.8278). Combined, about +0.05 AUC.

8.8.3 Real JPEG Clean Images Expose a Deployment Problem - and the Fix

The v2 logistic-regression model scored well inside its training distribution but flagged almost every real JPEG clean photo as stego with probability 1.0: SRM residuals react strongly to JPEG high-frequency noise. The fix was to add 414 real JPEG clean photos to the training corpus, producing `dataset_campus_v2_jpeg.csv`, then grid-tune LightGBM. [Corrected 2026-09-14] Those rows were originally given independent photo_ids while their features were bit-identical to the matching clean row (a duplicate), so a source photo could straddle the train/validation split and the by-photo discipline was nominal only; the same corpus was also built with GPU features that scaled pixels by 1/255 before the high-pass filters, differing from the CPU inference path by ~30x. Both are fixed and rerun (see `experiments/add_jpeg_clean.py`, `docs/RESULTS.md`, `CHANGELOG 1.6.2`). Current numbers: 143-D held-out 0.8939, 8-split 0.8980, weak nsF5 p3 d0.25 detection 50.0%; 53-D 0.8391 / 0.8461 / 41.4%.

8.8.4 Dual-version Deployment: 143d Robust vs. 53d Interpretable

Interpretability, recomputed after the audit: within the 143-D set the 90 SRM dimensions carry 52.6% of the LightGBM gain (BASE 11: 20.7%, LSB PREFIX 20: 20.6%, PREFIX 20: 6.2%); ablation shows that dropping them costs 0.05 AUC (5-fold OOF 0.9010 -> 0.8513). SRM is not a noise block once the scale is right. The project therefore ships both:

Dimension	143d default (robust)	53d interpretable
Features	BASE11 + SRM90 + PREFIX20 + LSB-PREFIX20 + TEX/EST2	Same minus SRM90 (53 D)
8-split mean AUC	0.8980	0.8461
Held-out AUC	0.8939	0.8391
Weak nsF5 p3 d0.25	50.0%	41.4%
Real-JPEG clean OOD	9.58% (n=1514, 95% CI 8.20-11.16%)	28.86% (n=1514, 95% CI 26.64-31.20%)
Best for	Deployment / mixed domains / real images	Teaching / defenses / per-image explanations (AUC ~0.05 lower)

`MLPredictor()` loads the 143d default; pass `model_path` to switch to the 53d model.

Prediction automatically selects 11-D or 143-D features from the model metadata. GUI model switching is still planned; for now, switch through the API:

Switching between the dual models

```
from ml_predict import MLPredictor

# default: 143d robust model (models/stego_classifier.joblib)
pred_143 = MLPredictor()

# 53d interpretable model (SRM 90 removed; higher AUC)
pred_53 = MLPredictor(
    model_path='models/stego_classifier_v2_jpeg_lgb_51d.joblib',
    clip_outliers=False)

r = pred_53.predict(img)
# {'probability': ..., 'verdict': ..., 'threshold': ...}
```

Try it | Experiment: train on `dataset_campus_v2_jpeg.csv` (set `$env:DS_FILES`), then compare 143d vs 53d on real JPEG clean photos and on weak nsF5 p3 d0.25 images. Put the AUC and OOD behavior into your report.

Watch out | The v2/53d models are only reliable near their training distribution: campus-derived PGM/BMP grayscale plus the added JPEG clean photos. For a brand-new camera or compression source, run a small OOD test before deployment.

8.9 Summary and Self-Check

- Under small samples and weak signals, domain features + simple models usually beat raw-pixel CNNs;
- The 11 v1 features = RS family + chi-square family + entropy/randomness baselines;
- Training/calibration/test separation plus GroupKFold is the skeleton of experimental trust;
- ML probability should be cross-checked with heuristics; weak densities have physical limits;
- Since v1.4, dual models are strong inside their distribution, but real-JPEG OOD validation remains mandatory: on 1,514 real clean photos (414 campus + 100 DIV2K + 1,000 ALASKA#2) the 143-D model false-positives 9.58% (95% CI 8.20-11.16%) and the 53-D model 28.86% (26.64-31.20%); 2K DIV2K images are the main failure source (~51%). Script: `experiments/ood_eval.py`; results: `docs/RESULTS.md` section 5.

Think about it | Seven samples come from one photo. Why is a random 80/20 split not acceptable? Design a minimal example showing how much leakage can inflate AUC.

Chapter 9 - Engineering: C++, GPU, GUI, and Tests (Week 10)

Try it | Goals: see how algorithms become software; understand why C++ and GPU acceleration exist and how cross-language consistency is guaranteed; run the tests and trace the CI.

9.1 Architecture: Layered, Not Coupled

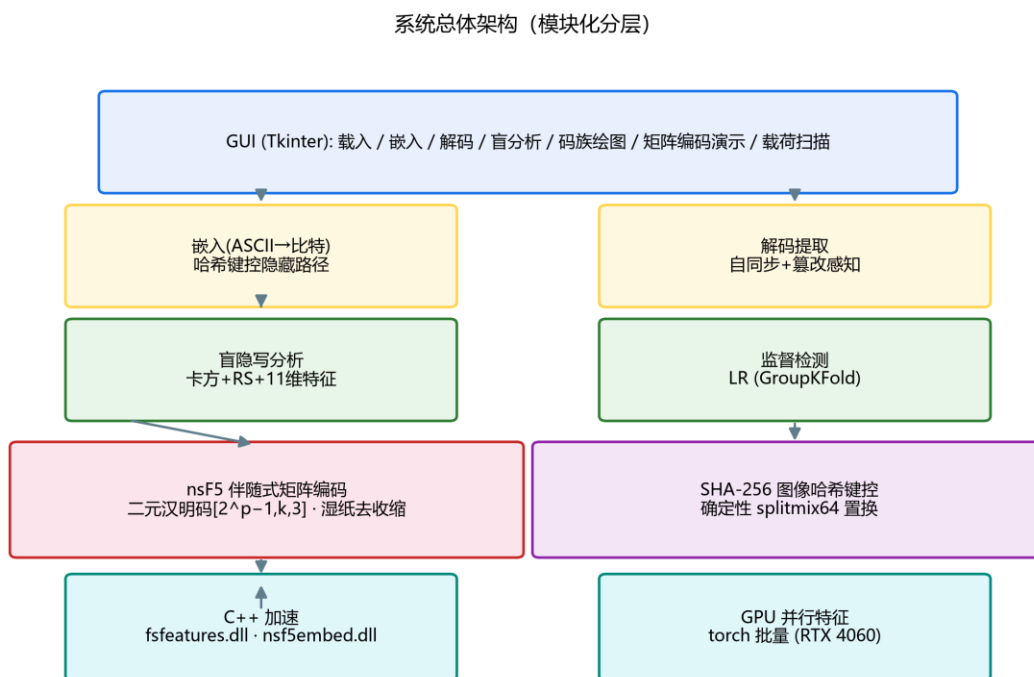


Fig. 9-1 System architecture: acceleration layer -> algorithm/security core -> features -> GUI (project figure)

- **Acceleration layer:** cpp/fsfeatures.dll (features), cpp/nsf5embed.dll (embedding/shuffling);
- **Algorithm core:** ns5_core.py - Hamming codes, wet paper, hash keying, embed/extract;

- **Feature layer:** steganalysis.py, efficiency.py, make_dataset.py, train_model.py, ml_predict.py, featurize_v2.py, srm_filter.py;
- **UI layer:** gui.py + matrix_demo.py + scan_panel.py - one-click access to everything.

Layering matters because the algorithm layer does not depend on the GUI (it runs on servers and in CI), and the acceleration layer falls back to pure Python when a DLL is missing, so the features always work.

9.2 C++ Acceleration: Hot Paths and How We Know They Are Right

Python is slow at elementwise loops, and this project has two classic hot paths:

deterministic permutation (shuffling up to 16 million positions) and **feature extraction** (RS/chi-square/entropy statistics per image). Moving them into C++ DLLs gives orders-of-magnitude gains: permutation speedup grows with N (data: experiments/data/bench_permute.csv, reproduce with ``python experiments/tools/gen_bench.py --only permute``; timings are machine-dependent): at 4096^2 (16M positions) Python ~15.6 s -> C++ ~0.24 s (~65x), while the small-N end of the same benchmark peaks around ~240x (65k).

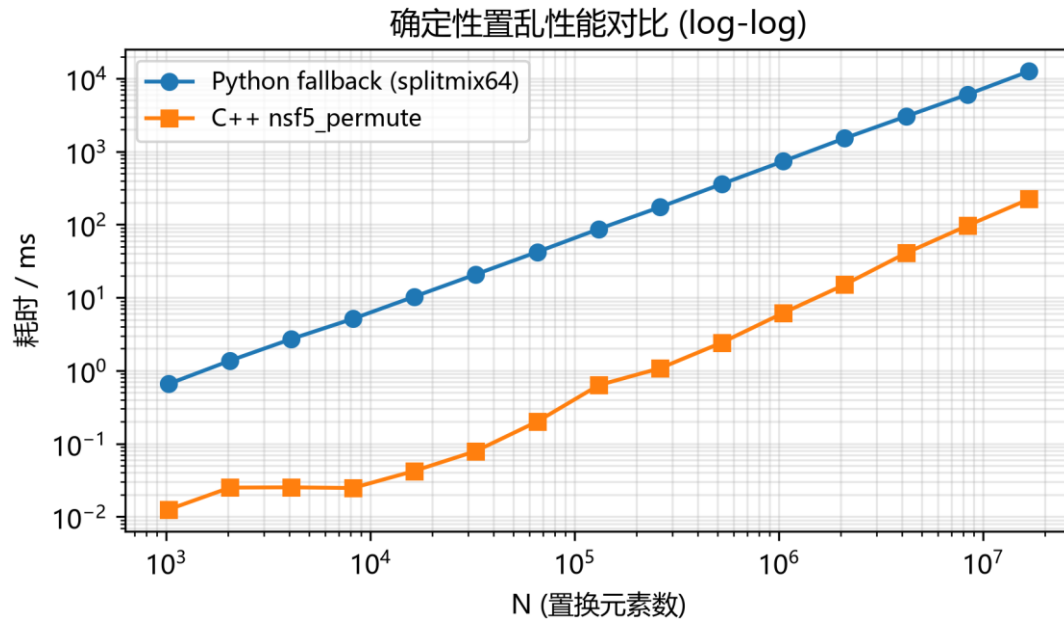


Fig. 9-2 Deterministic permutation: Python vs C++ (log-log axes, project figure)

Speed is worthless unless the result is identical. Python calls the DLLs through ctypes, and the project guards correctness with self-checks:

- `cppembed.py` embeds the same image through C++ and Python paths and compares outputs pixel by pixel;
- `fsfeatures.py` `check_against_python()` compares every C++ feature with the Python reference;
- When the DLL is missing or fails to load, the code falls back to the same Python algorithm, keeping embed/decode reversible everywhere.

Watch out | Cross-language consistency is the prerequisite for acceleration and a classic debugging minefield: bitwise operations, rounding, and overflow differ between C++ and NumPy. If you ever see "embeds with DLL, cannot decode without it," run `cppembed.selfcheck()` and `fsfeatures.check_against_python()` before touching the algorithm.

9.3 GPU Version: Batch-Vectorizing Features

The `gpu/` directory rewrites feature computation as PyTorch tensor operators: load a batch of 512x512 grayscale images, compute histograms, RS, chi-square, and entropies in parallel on the GPU, then return results to the CPU. v1.4 adds `gpu/featurize_v2_gpu.py` for the 143-D v2 features (including SRM convolutions). The v1 pipeline extracts 2,070 images in about 5 seconds (~410 images/s) and matches the CPU reference bit-for-bit (float error ~1e-7).

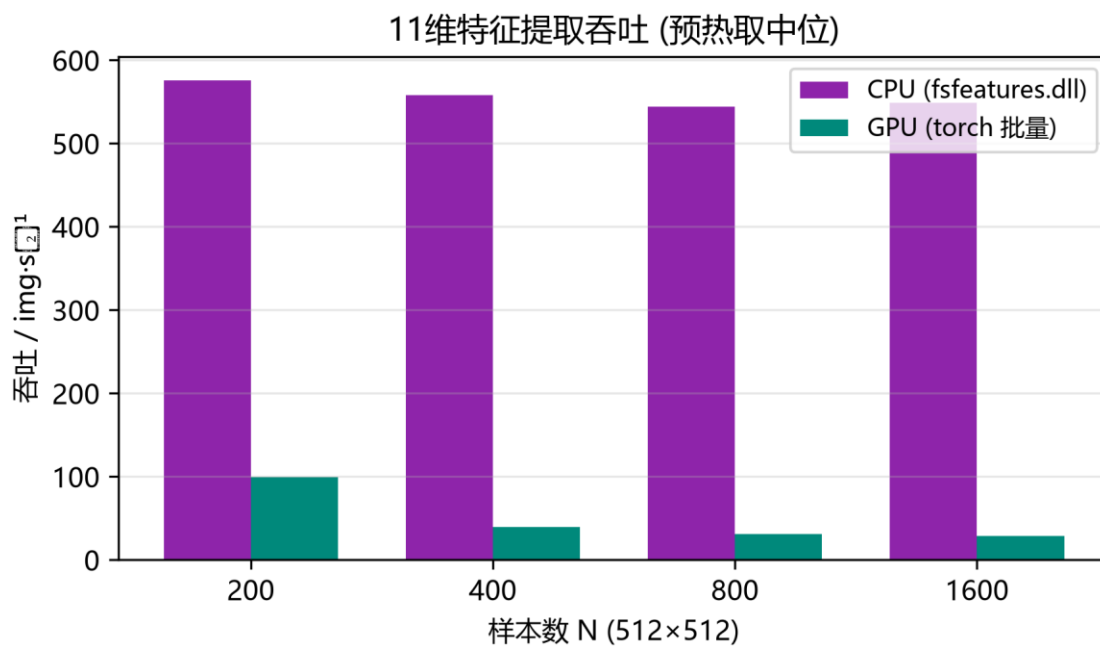


Fig. 9-3 v1 11-D feature throughput: CPU (C++) vs GPU (torch batch) (project figure)

Read the code | Read the self-check in `gpu/featurize_gpu.py`: it compares GPU results element by element with `src/fsfeatures.py`. Bit-level consistency is what makes the dual implementation safe. Then read the README discussion of throughput limits: for small batches, host-device transfer can make GPU slower than C++ - an honest measurement, not a marketing claim.

9.4 GUI: Turning a Research Tool into a Teaching Application

`src/gui.py` is organized around Embed -> Decode -> Analyze. Two teaching panels stand out:

- **Matrix coding demo** (`matrix_demo.py`): click a block, watch s , m , and d update live, see the matched H column highlighted, and verify $H \cdot x' = m$ after the flip;
- **Payload scan** (`scan_panel.py`): drag payload from 0 to 0.4 and watch chi-square p , RS estimate, and ML probability curves refresh as the image is re-embedded at each density.

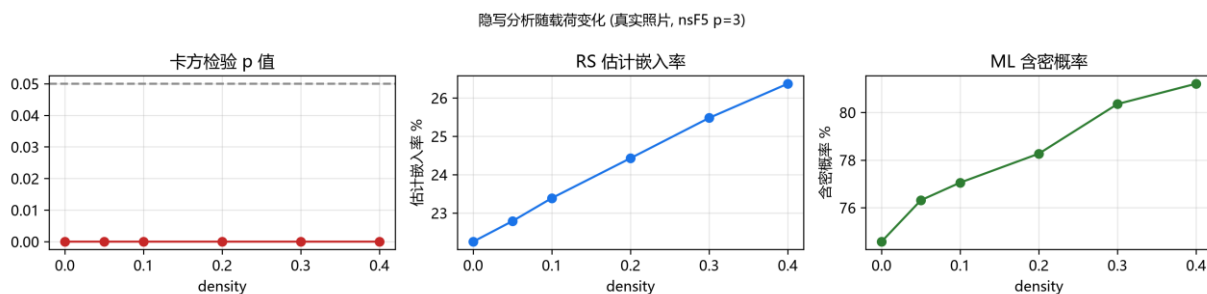


Fig. 9-4 Payload scan: detectability grows with embedding density (project figure)

Watch out | Note the wording in the GUI: a single image has no true "AUC" (AUC needs a set of positive and negative samples), so the scan panel plots model probability as a trend illustration. Distinguish "single-image probability" from "dataset metrics" whenever you read software output.

9.5 Tests and CI: Refactor Without Regret

Test file	Verifies	Mental model
<code>test_core.py</code>	Hamming matrices, embed/decode round trips, wet paper, wrong passwords	Algorithms still work
<code>test_steg.py</code>	Blind analysis separates clean from stego	Analysis still works

Test file	Verifies	Mental model
test_false_positive.py	Clean images are not flagged (regression)	False positives stay controlled
test_gui.py	Window builds, loads images, previews	UI still works

`.github/workflows/ci.yml` runs core tests and builds wheel + sdist on every push/PR to main; pushing a `v*` tag creates a GitHub Release. Tests + packaging + auto-release is the standard way an algorithm project becomes a reusable tool.

9.6 A Code-Reading Route (Use as Needed)

1. Entry point: `src/run_e2e.py` - the shortest path through every module;
2. Data layer: `image_io.py` - how arrays enter and leave;
3. Embedding layer: read `ns5_core.py` top to bottom (hash, shuffle, Hamming, wet paper, high-level API);
4. Analysis layer: trace `analyze()` in `steganalysis.py` backwards through each statistic;
5. ML layer: `train_model.py` main flow -> `featurize_v2.py` / `srm_filter.py` (v1.4) -> `ml_predict.py` dual-model inference;
6. Acceleration layer: `cppembed.py` / `fsfeatures.py` ctypes bindings and self-checks;
7. UI layer: find a button callback in `gui.py` and follow it to the algorithm function.

Think about it | Why does the comment in `ns5_core.py` insist that changing the permutation algorithm breaks reversibility? Combine with the v1.2.1 fix (automatic Python fallback): what catastrophe happens if Python and C++ permutations differ?

9.7 v1.4.0 Update: SRM, Multi-Source Data, and Model Engineering (2026-09)

Several engineering modules changed in v1.4.0. Learn the new directory first:

New file / module	Role	What to study
src/srm_filter.py	30 standard SRM high-pass kernels; numpy/torch dual implementations	Kernel list, normalization, enhanced-image pipeline
src/featurize_v2.py	143-D v2 feature assembly on CPU	ALL_FEATURE_NAMES, featurize_v2()
gpu/featurize_v2_gpu.py	GPU batch 143-D features and consistency self-check	extract_features_v2_gpu()
src/make_dataset.py extensions	Multiprocessing -j, SRM/feature- set/variants options	Per-image parallelism, identical row output
src/train_model.py extensions	DS_FILES multi-dataset, stacking, LGB tuning	Dual-model saving and thresholds
gpu/make_imageset.py extensions	memmap disk write, --out/--id-offset	Multi-source partitioning without OOM

On the data side, v1.4 established a pure campus-photo baseline data/campus_jpg (414 photos; earlier DIP4E textbook images were removed) and added the standard steganalysis benchmark BOSSbase 1.01 (10,000 512x512 grayscale PGM images) for multi-source training. Re-checked 2026-09-14

(experiments/data/gpu_pipeline_metrics.csv): GPU campus-only AUC 0.7903 (Youden 0.713, acc 0.802), BOSSbase-only AUC 0.6438 (0.798 / 0.655); the merged run uses the 10,000-sample BOSSbase imageset available in this repo (the older number used 50,000), so it was rerun as 3,410 + 10,000 = 13,410 samples with AUC 0.6672. make_dataset.py now supports multiprocessing (~5.6x on 16 cores) and the GPU imagesets are written per-image with memmap to avoid large-array OOM.

Tooling changed too: the license moved from MIT to Apache-2.0 with a NOTICE file and third-party attributions, and README/pyproject now declare v1.4.0. The pixel-level and feature-level self-checks between C++ and Python remain in place.

Try it | Run `python gpu\featurize_v2_gpu.py self-check`. Then follow the README to generate one source from `data\campus.jpg` and one from `data\BOSSbase_1.01`, and compare three models: campus only, BOSSbase only, and merged. That is the most instructive data-domain experiment in v1.4.

Chapter 10 - Capstone: From Reading to Building (Weeks 11-12)

Try it | Goals: complete one demonstrable improvement experiment in two weeks. Three directions are given below, easy to hard; you may combine them.

10.1 Direction A: Reproduce and Critically Verify (Safest)

Choose one or two claims from the README / docs/RESULTS.md, rerun them, and try to break them:

- Reproduce the code-family figure: does measured $\alpha(p)$ match theory? Where does deviation creep in at large p ?
- Reproduce detection with your own photos: compare the v1 11-D baseline (~ 0.75 - 0.79) with the README's v1.4 dual models, and see how photo style moves AUC;
- Reproduce the tamper experiment: flip 1 pixel, 1 row, 1 block - what are the failure rates?

Acceptance item	Passing standard
Experiment log	Environment, commands, parameters, raw outputs recorded
Figures	At least two plots with complete axes and legends
Conclusions	Distinguish "confirms the paper" from "disagrees with the paper" with reasons
Reproducibility	One-click script or a list of commands others can rerun

10.2 Direction B: Feature Extensions (Most Satisfying)

- **UTF-8 / Chinese messages:** is already implemented in `encode_string()` since v1.8 (the length header counts bytes, not characters); compare your own version with the source;
- **Color channels:** only the R channel is used today; study how to distribute capacity and security across R/G/B;
- **New features:** add a statistic you believe separates clean/stego on top of v1 11-D or v2 143-D, and measure with GroupKFold whether AUC really rises;
- **Baseline comparisons:** implement naive LSB replacement and compare Gn collapse and detection rates against nsF5 at equal payload.

Watch out | After every extension, run `test_core.py` and `test_steg.py`: embed -> decode must still round trip. When you change the encoding scheme, the decoder must change with it - the most commonly forgotten step.

10.3 Direction C: Repackage a Teaching Demo

Turn the GUI matrix-coding demo into a self-contained lesson or animated script, e.g.:

- Step animation: random block -> show H -> compute s -> give m -> compute d -> highlight -> verify;
- Quiz mode: generate random problems asking "which position flips?" with automatic scoring;
- Compare mode: embed the same message with LSB, matrix p3, and nsF5 p3, showing changes and Gn side by side.

10.4 Document-to-Code Reading Table

Note: this repository no longer ships the paper manuscripts; every reported number now lives in docs/RESULTS.md (the canonical results table), which labels each row with its corpus, protocol and traceability. The table below keeps the early paper's structure as an example - cross-reference that table.

Paper section	Corresponding code	Prereqs before reading
Ch. 2 Background	ns5_core.py / steganalysis.py	LSB, Hamming, wet paper, chi-square/RS
Ch. 3 Requirements	README feature overview	Steganography vs steganalysis game view
Ch. 4 Design	architecture figure + ns5_core.py	Layers, hash-keyed pipeline
Ch. 4 Acceleration	cppembed.py / fsfeatures.py	ctypes and consistency self-checks
Ch. 4 GPU	gpu/*.py	Tensor ops, batching, throughput limits
Ch. 5 Experiments	run_e2e / make_dataset / train_model	GroupKFold, AUC, Youden

10.5 Common Defense / Presentation Questions

Frequent question	How to answer
Why is LSB hiding detectable?	Pair counts flatten (chi-square) and LSB structure collapses (RS); intuition first, statistics second
Why does matrix embedding change less?	$n=2^p-1$ positions carry p bits; distinct H columns locate every syndrome difference
What does nsF5 improve over F5?	Wet paper marks wet positions in advance and solves on dry ones - no shrinkage, no retries
How is the wet-paper equation solved?	GF(2) system $H_{\text{dry}}y=d$; single column, then pairs, then Gaussian elimination
Why not a raw-pixel CNN?	Weak signals plus tiny independent-sample counts; features + simple models generalized better

Frequent question	How to answer
What does AUC 0.75 vs 0.91 mean?	Ranking ability; AUC is threshold-independent while deployed detection needs an operating point
What does hash keying protect against?	Predictable/copyable positions and naive tampering; it is not a keyed MAC
Any risk from C++/GPU acceleration?	Yes - cross-language consistency checks are mandatory or embed/decode breaks
Why keep 143d and 53d models?	143d is more robust OOD; 53d has higher AUC and full interpretability - different jobs

10.6 Delivery Checklist (Final Week)

- A code diff you can explain change by change;
- Experiment scripts plus raw outputs (no cherry-picking);
- Two or three figures (efficiency / ROC / density / tamper);
- A 3-5 page report: problem -> method -> results -> limits -> next steps;
- A 5-minute live demo from embedding to analysis;
- Ability to answer any three questions from 10.5 without notes.

Chapter 11 - Boundaries, Ethics, and What Comes Next (Read Along the Way)

11.1 Legal and Ethical Boundaries

Steganography is double-edged: it supports watermarking, media provenance, and covert authentication, but it can also be abused for malicious covert communication. The right posture for learning and research is:

- Experiment only on your own data, public datasets, or authorized material;
- Use the tooling for detection and forensics rather than concealment;
- Report limitations and false-positive risk honestly; never overstate detection power;
- Do not use other people's private photos as practice carriers; de-identify and authorize data.

Watch out | The steganalysis half of this project (chi-square/RS/ML) is the shield: monitoring and forensics need it to find covert channels. Think about your data and purpose before studying attack surfaces.

11.2 Known Limitations (Understanding Boundaries Is Understanding the Project)

- Messages support UTF-8 by default (Chinese works out of the box);
- Color images use only the R channel, underutilizing capacity;
- Keying is not a keyed MAC, so a strong attacker who re-embeds can update the header hash;

- The v1 dataset still starts from a limited set of independent photos (414 campus, later expanded with BOSSbase); weak-density and cross-source detection remain hard;
- The v2/143-D model was over-aggressive on out-of-distribution real JPEG clean images; the fix requires JPEG clean samples in training - always run OOD tests before deployment (current numbers: 1,514 real clean photos, 143-D 9.58% / 53-D 28.86% false positives);
- SRM residuals are the single largest gain contributor (52.6% of LightGBM gain) once the feature scale is consistent, but they are also the least interpretable block - removing them is what makes the 53-D model explainable, at a cost of about 0.05 AUC;
- The dual models trade accuracy for interpretability (143-D: 8-split mean AUC 0.8980, weak-band detection 50.0%, OOD false-positive 9.58%; 53-D: 0.8461 / 41.4% / 28.86%, but every dimension is explainable); GUI switching is planned, use the API today;
- Heuristic analysis outputs a "stego tendency probability," not a strict statistical probability;
- Pixel-domain nsF5 is incompatible with lossy formats; JPEG-domain work needs the yccstego-style DCT coefficient design (companion project: <https://github.com/Yukinoshita-lin/yccstego>).

11.3 A Map of Modern Information Hiding and Steganalysis

Direction	Representative ideas / methods	Distance from an undergrad
Content-adaptive embedding	HUGO / WOW / UNIWARD: hide where texture is complex	Requires optimization background

Direction	Representative ideas / methods	Distance from an undergrad
Deep-learning steganography	Encoder-decoder models trained end-to-end	Reproducible entry projects exist
Deep-learning steganalysis	SRNet / large-scale CNNs	Needs big data and compute
JPEG-domain hiding	yccstego: nsF5 on quantized DCT coefficients of Y (companion project: https://github.com/Yukinoshita-lin/yccstego) (companion project: https://github.com/Yukinoshita-lin/yccstego)	Direct extension of this project
Robust/reversible watermarking	Trade invisibility, robustness, capacity	Engineering-heavy; industry-relevant

If your interest is sparked, the next steps are concrete: read the original wet-paper paper (Fridrich et al.), study the companion project yccstego's quantized-DCT pipeline (<https://github.com/Yukinoshita-lin/yccstego>), then pick one modern algorithm for a literature review - Appendix E gives entry points.

11.4 Closing Words

At the start, steganography may have sounded like "hiding words in images." Now you can explain why it depends on carrier redundancy, why direct LSB changes leave statistical fingerprints, how Hamming codes carry the most information per modification, how wet paper coding removes shrinkage, and how machine learning states its confidence honestly. Those ideas connect one project in F:\Steganography to a way of thinking shared by information security, probability, and machine learning.

Try it | Final exercise: without notes, write 500 words explaining to a classmate why nsF5 is better than naive LSB. Then open the README and compare. If you can write it, you have finished.

Chapter 12 - Critical Thinking (A Post-Mortem Guide)

Tip | Read the questions first, then the text. **Core question: once the project is done, how do you make sure the experiments and the narrative have not fooled you?**

By the end of this chapter you can:

- Retell the full story of the source-image leakage incident (incident -> localization -> fix -> conclusion overturned), and say what "trust the experiments" actually means;
- Audit a conclusion with the five-question checklist, each item closing the "question -> verify -> conclude" loop;
- Turn a piece of wishful thinking into a plan that passes: **executable, falsifiable, with a control**;
- Read "conclusions" and "evidence" separately, knowing that a single counterexample suffices to overturn a conclusion;
- Write "known limitations" and "open problems" into a report, and recite the five elements of an experiment log.

This chapter teaches no new algorithms. What it teaches is how to make what you have already built survive someone else re-running it.

Opening question: what makes a conclusion safe to say out loud? Over the previous eleven chapters you learned "how to hide things" and "how to find them", and in Chapter 10 you built a capstone project with your own hands. But done is not the same as done right - **the person an experiment fools most easily is the one who ran it**. This chapter is a post-mortem guide with eight takeaways, all taught through incidents that actually happened in this project: a source-image leakage that inflated AUC by 0.14, a "deep learning of course is stronger" narrative refuted by a single 0.5007, and an "I feel it helps" rewritten as an ablation with a control and a falsification condition. After this chapter, you should be able to upgrade "I understood it" into "I dare let someone else re-run it".

Section	Question it answers	Your output
12.1 Trust the experiments	How do experiments fool you?	You can retell the full source-image leakage incident
12.2 Question the design	What should you ask when handed a conclusion?	You can use the five-question checklist
12.3 Propose a plan	How do you turn questioning into a plan?	You can write executable / falsifiable / controlled plans
12.4 Conclusions vs. evidence	How do you keep conclusion and evidence apart?	You know one counterexample can overturn a conclusion
12.5 Admit you don't know	Why is "I don't know" a skill?	You can write "known limitations" and "open problems"
12.6 Beware the narrative	Which "of course"s are traps?	You can cite three counterexamples from the project
12.7 Reproduction first	When reproduction fails, whom do you suspect first?	You remember the environment -> understanding -> book order
12.8 Experiment logs	What makes a conclusion traceable?	You can write the five log elements

12.1 The Experimental Results Are the Final Word: How Experiments Fool You

The point of this section is not "experiments matter" but **"how experiments fool you"**. Start with a numeric conflict that really happened: the same 143-D model scores **AUC 0.8062** on the BOSSbase corpus but **0.8939** on the self-built campus corpus. The two numbers are far apart, yet this is no contradiction - **the corpora have different**

properties. BOSSbase is the domain-standard benchmark and hard; the campus corpus is self-built, contains real JPEG clean images, and is easier than BOSSbase. Put the two corpora's numbers side by side and what you have compared is not "the model got stronger" but "you changed the exam".

A more serious class of deception happens in **the experimental design itself**. One incident really occurred in this project: the `clean_jpeg` rows of the campus corpus wrongly used independent `photo_ids` - 414 duplicate rows each got their own id, and the "split by source image" discipline was silently broken: the clean and stego versions of the same source image landed on opposite sides of the train/test split. As soon as the model recognized "which source image this is", it could infer the label too. The result: **143d AUC was lifted from 0.7555 to 0.8946**, and weak-band detection inflated from **44.2% to 85.3%**. The model did not get stronger - **the experimental design was wrong**.

Stage	What happened	Key action
Incident	414 duplicate <code>clean_jpeg</code> rows each held a <code>photo_id</code>	Same-source copies straddled the train/test sides
Localization	The "split by source image" invariant check failed	Re-split by source image, recompute confidence intervals
Fix	Generator and trainer both reject invariant-breaking corpora before training	Turn the invariant into an executable check
Overturn	143d AUC 0.8946 -> 0.7555; weak-band detection 85.3% -> 44.2%	Old conclusion voided; documents rewritten after the re-run

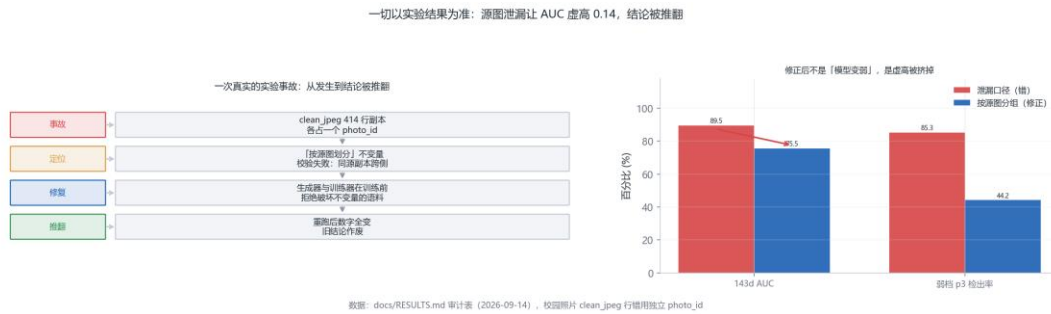


Figure 12-1 The source-image leakage incident and the overturned conclusion. This figure answers: how one experimental-design error inflated AUC by 0.14 and ultimately overturned the old conclusion. Left: the four stages incident -> localization -> fix -> overturn; right: the old numbers and the corrected ones side by side

Try it | Open StegoLab's `tools/run_ch12.py` and run it. It uses a **synthetic corpus** to demonstrate the leakage mechanism live: the label is decided only by "source-image identity", and the features carry source-image fingerprints. With a random per-image split the AUC is about **0.74**; grouped by source image it falls back to about **0.52** - inflated by about **+0.22**. In your own terminal you will watch "the way you split" conjure 0.2 of score out of thin air. The model did not get stronger; the design was wrong.

So **every conclusion must carry qualifiers**: corpus, protocol, confidence interval, traceability. Numbers without these qualifiers cannot be cited as conclusions. The project turned this discipline into an executable check: the authoritative results table `docs/RESULTS.md` is generated by the script `experiments/build_results_table.py`, and every number must be traceable to "script + CSV"; anything untraceable goes into Section 7 with the reason annotated.

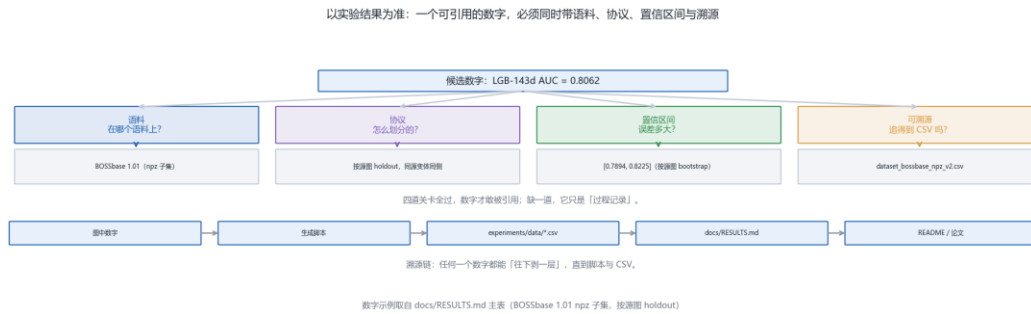


Figure 12-2 The four gates a citable number must pass. This figure answers: which numbers you may dare to cite - corpus, protocol, confidence interval, and traceability, all indispensable. Top: the four gates and the question each one asks; bottom: the provenance chain from a number in a figure all the way down to scripts and CSVs

Watch out | The greatest danger is not "no numbers" but "**beautiful numbers without qualifiers**". The sentence "AUC 0.89" means nothing on its own - it does not say which corpus, which split, how large the error is, or whether it is traceable. If, at the defense, someone presses you with "under which protocol was that 0.89 measured?" and you cannot answer, every bit of work before it loses credit.

12.2 Dare to Question the Design, but Ground It in Thinking

Questioning is not nitpicking. Nitpicking stops at "I feel this is wrong"; questioning must come back down to "**I actually verified it, and the result is ...**". The project offers plenty of ready targets for questioning, and the checklist below pairs each one with a real verification action and its measured result - what you see is not opinion but the complete loop of "question -> verify -> conclude".



Figure 12-3 The question checklist and the verification loop. This figure answers: which five questions to ask when handed a conclusion, and the verification action and measured result for each. The five items: data provenance, fairness of the control group, correlation or causation, simpler explanations, and whether the metric answers the question you meant to ask

Question	Verification action	Measured result in the project
On what data was this conclusion obtained? Does it still hold on another batch?	Re-run the same model on a different corpus	BOSSbase 0.8062 vs campus 0.8939 - different corpora, the numbers are not comparable
Is the control group set up fairly?	Check whether both sides share the same source-image split	The CNN and LGB baselines share the split - that is what makes them comparable
Has "correlation" been passed off as "causation"?	Check whether an ablation was run	SRM ablation $\Delta = +0.0497$ - causal evidence, not mere correlation
Has a simpler explanation been ignored?	First check whether training converged	Bare CNN training-set AUC 0.5005 (not converged) - not "CNNs are weak"
Can this metric answer the question I actually have?	Look at weak-band detection, not just AUC	High AUC $\neq$ usable at weak densities (weak-band detection drops noticeably)

Think about it | "The bare CNN scores only 0.5007 on BOSSbase" is often cited as evidence

that "deep learning is worse than hand-crafted features". But first ask the simpler question: **did this model's training converge?** One check settles it - its training-set AUC is only 0.5005, indistinguishable from random guessing. When an unconverged model loses the contest, what it shows is "this training run failed", not "this road is a dead end". The gap between "correlation" and "causation" is often just this one question.

12.3 After Questioning, Dare to Propose Your Own Plan - and Propose It Well

Those who can only question stay where they stand; those who can do research take one step further - **they propose their own plan**. Here you must tell "wishful thinking" apart from a "plan":

Wishful thinking says: "I feel this algorithm is no good." A plan says: "If we replace X with Y, then on data Z we should see change W, because ...; I designed the following experiment to verify it." The difference lies not in tone but in three hard criteria: **executable, falsifiable, with a control**.

What "well" means	Concrete requirement	Counterexample (fails)
Executable	Can be finished within limited time and compute	"Retrain a billion-parameter model"
Falsifiable	You can write down in advance "what result would overturn my hypothesis"	"If it doesn't work well, I'll tune it some more"
Controlled	State clearly what changed, compared against what, and under which protocol	"I changed something and it feels better"

The project contains a complete example of walking from "question" to "plan": the starting point is one question - "do the SRM features actually help?" The endpoint is not a feeling but an **ablation experiment**: remove the SRM group from the 143 dimensions (falling back to 53) and run 8 seeds on the campus corpus with source-image GroupKFold(5). The result: **OOF AUC drops from 0.9010 to 0.8513** ($\Delta = -0.0497$), and 8/8 seeds agree. That is a hundred times stronger than "I feel SRM helps": it has a control, a protocol, and a falsification condition.



Figure 12-4 From wishful thinking to a plan. This figure answers: how to rewrite an "I feel" into an executable, falsifiable, controlled plan, with the SRM ablation as the real example. Top: wishful thinking and plan side by side + the three hard criteria; bottom: the ablation's measured answer and its falsification condition

Try it | Write a "plan card" for your own topic, and fill all four fields: **hypothesis** (one sentence), **control** (compared against what, under which protocol), **falsification condition** (what result would make you concede), and **cost** (how long it runs, whether a GPU is needed). If any field stays empty, it is still wishful thinking. Here is how the project filled the SRM ablation card: hypothesis "the SRM group improves OOF AUC"; control "143d vs 53d with SRM removed, the same 5-fold GroupKFold"; falsification condition "if removing SRM does not lower AUC ($\Delta \leq 0$), the hypothesis is overturned"; cost "8 seeds $\times$ 5 folds, minutes-scale".

12.4 Distinguish "Conclusions" from "Evidence"

When reading papers and writing reports, the two things most easily conflated are the **conclusion** and the **evidence that supports it**. The conclusion is one sentence: "SRM

features help." The evidence is another sentence: "under a fixed protocol, adding SRM raises OOF AUC by +0.0497, and 8/8 seeds agree." A conclusion can rest on many pieces of evidence, and it can also **be overturned by a single counterexample**. This section's task is to teach you to keep the two apart, always.

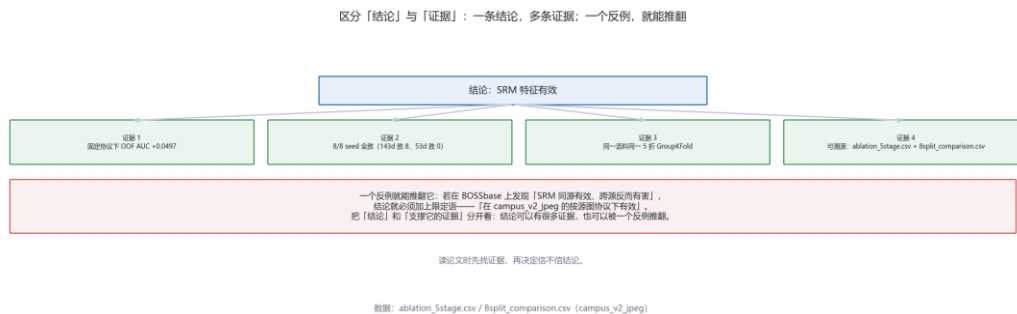


Figure 12-5 Separating conclusion from evidence. This figure answers: which pieces of evidence hang under one conclusion, and why a single counterexample suffices to overturn it. Top: the conclusion "SRM features help" and its four pieces of evidence; bottom: how a counterexample forces the conclusion to take on qualifiers

For example: if one day BOSSbase shows that "SRM helps same-source but is **harmful cross-source**", then "SRM features help" must gain a qualifier - "helps under the campus corpus's split-by-source protocol". That is not an embarrassment; that is science: **a conclusion's scope of validity is decided by the boundary of its evidence**. So when you read any paper, find the evidence first (what data, which protocol, how many seeds) and only then decide whether to believe the conclusion.

Tip | A practical exercise: whenever you meet a conclusion sentence, list right there beside it "which pieces of evidence it needs at minimum to hold". If you cannot list them, you have not really understood it; if you list them and find the evidence insufficient, the author overclaimed. Every table in Appendix K does exactly this - **conclusions in the main text, evidence in the appendix, a one-to-one match between them**.

12.5 Admitting "I Don't Know" Is a Skill

One of the most valuable parts of this project is its **correction record**. It shows that admitting you were wrong, admitting a number cannot be trusted, and admitting some problems remain open is nothing to be ashamed of - it makes the whole project more credible. The source-image leakage incident of Section 12.1 was not "deleted and rewritten" out of the documents; it was kept as-is in the audit records - because **one honestly recorded failure is worth more than ten vague successes**.

"I don't know" is not an endpoint; it is the starting point of the next experiment. For writing, this becomes two hard requirements: an experiment report **must contain a section on "known limitations" and "open problems"**, and in a defense, **admitting limitations earns more trust than pretending perfection**. The three known limitations listed in the project's Chapter 11 (color images use only the R channel, keying is a hash rather than a MAC, pixel-domain work does not survive JPEG) each state "why it was acceptable at the time" and "the right way to upgrade" - that is the standard way to write it.

Watch out | "Known limitations" is not a self-criticism list; it is a **specification of boundaries**. Stating clearly "under which conditions I do not guarantee it works" is precisely an endorsement of "under which conditions I guarantee it works". Conversely, a tool that claims "works everywhere" is the most suspicious of all - either it never tested its boundaries, or it never dared write them down.

12.6 Beware "Plausible-Sounding Narratives"

Many wrong conclusions come not from faked data but from **narratives that flow too smoothly**. "Deep learning of course beats hand-crafted features", "more features of course help", "better on the training set, so of course better on the test set" - all three

sound self-evident, yet in this project each has a ready-made counterexample. Part of critical thinking is staying alert to whatever "sounds self-evident".



Figure 12-6 Plausible-sounding narratives and their counterexamples. This figure answers: which project counterexample breaks each of the three "self-evident" narratives. Top: the narrative; middle: the project counterexample; right: the lesson - strength depends on preprocessing and convergence, adding features requires proving they help first, and a wrong split makes the score fake

Sounds self-evident	Counterexample in the project	Lesson
"Deep learning of course beats hand-crafted features"	Bare CNN 0.5007 (training AUC 0.5005, not converged); fixed SRM + small CNN 0.9541; hand-crafted LGB-143d 0.8062	Strength depends on preprocessing and convergence
"More features of course help"	11d 0.8708 -> 31d 0.8665 -> 51d 0.8516 -> 53d 0.8513 -> 143d 0.9010 (the first four steps go down, not up)	Adding features requires proving they help first (ablation)
"Better on the training set, so of course better on the test set"	Source-image leakage: train/test shared source-image fingerprints; AUC 0.8946 was inflated, 0.7555 after the fix	With a wrong split, the score is fake

Think about it | The curve in the second row deserves a long look: from 11 dimensions up to 53, AUC does not rise but **falls** (0.8708 -> 0.8513), and jumps to 0.9010 only when the SRM

group is added. Had you followed the "more features are better" intuition, you would have walked from bad to worse through the first four steps without noticing. **Intuition proposes hypotheses; experiments adjudicate** - keeping those two jobs apart is the entire point of this chapter.

12.7 Make "Reproduction" the First Lesson of Critical Thinking

Reproduction is not just "getting the code to run"; it means **re-verifying with your own data and your own protocol**. The companion StegoLab exists for exactly this: every key number in the handbook has a corresponding reproduction path - figures are registered in `figure_manifest.json` with source and SHA-256, experiments land on disk in `figures/chXX_results.csv`.

If you cannot reproduce a result, troubleshoot in strictly this order: **suspect the environment first, your understanding second, and the book last**. Environment (versions, fonts, dependencies, random seeds) -> understanding (are the criteria, protocol, and units consistent) -> book (is the handbook itself wrong). This order is not politeness; it is probability: the overwhelming majority of "cannot reproduce" cases are exhausted by the first two causes. A person who can honestly say "I cannot reproduce it; the cause might be ..." is far more reliable than one who only says "I understood it".



Figure 12-7 Reproduction and experiment logs. This figure answers: in what order to suspect causes when reproduction fails, and which five elements a sound experiment log records. Top: the three reproduction

questions (environment -> understanding -> book); middle: five log elements; bottom: why the correction record is the best teaching material

12.8 The Experiment Log Is the Vehicle of Critical Thinking

Critical thinking is not an attitude; it is a **record-keeping habit that can be inspected**. For every experiment, write down five things: **environment, command, parameters, raw output, date**. No processing of the "pick the pretty numbers" kind; conclusions must trace back to the concrete experiment records. The project already does this solidly: the CSVs under `experiments/data/` are the raw disk output, `docs/RESULTS.md` provides line-by-line provenance, and Appendix K matches the numbers cited in the handbook against them one by one - **any sentence can be peeled one layer further, down to scripts and CSVs**.

```
v = 200
# Skeleton of a sound experiment record (the project's actual practice)
Environment: python 3.10 / numpy 1.26 / matplotlib 3.8 / font Microsoft YaHei
Command: python tools/run_ch12.py --out figures/ch12_results.csv
Parameters: seed=20261005, n_src=120, k=5, d=32, ridge_lam=0.1
Output: figures/ch12_results.csv (per-row case/detail/value/ok/note)
Date: 2026-10-05
# Conclusions may cite only numbers from "Output", and must carry the four qualifiers
corpus/protocol/CI/provenance.
```

Read the code | The runnable experiment for this chapter lives in

`F:\StegoLab\chapters\ch12_critical.py` (figures + health-check logic), entry point `tools/run_ch12.py`. It does five things live: (1) a qualifier health check (row by row, does every entry of the authoritative results table carry all four qualifiers); (2) a leakage recomputation (the AUC gap between random splitting and grouping by source image on the synthetic corpus); (3) an ablation recheck (0.9010 vs 0.8513, 8/8 wins); (4) narrative counterexample checks (bare CNN 0.5007 and fixed SRM 0.9541); (5) a falsifiability check. Results land in `figures/ch12_results.csv`, and every number in this chapter can be reconciled against it item by item.

12.9 End-of-Chapter Exercise: Give a Conclusion a Health Check

This chapter's exercise is not "compute a number" but "**audit a sentence**". For each of the three conclusions below, write a health-check report of at most 200 words, pointing out which qualifiers it is missing, how it could be misread, and what additional experiment you would run to support or overturn it.

Conclusion (to be audited)	Hint: what to ask yourself first
"Our model reaches AUC 0.89 - great performance."	Corpus? Protocol? Confidence interval? Traceable? - 0.8939 is the campus corpus, source-image holdout result; does it still hold on another corpus?
"Adding SRM features improves performance."	What is the control? What protocol? How large is the gain? How many seeds? Is there a falsification condition?
"The detector is 95% accurate on the test set - deploy it."	How was the test set split? Any source-image leakage? What is the false-positive rate on real clean photos (OOD)?

Try it | Rewrite each of the three conclusions above into its "version with qualifiers", then check it against the real numbers in Appendix K. The rewrite is finished when: **whenever any sentence is pressed with "on what data, under what protocol?", you can answer at once and point to which row of which CSV it sits in.** Achieve that, and you have finished Chapter 12.

Chapter summary. Critical thinking does not ask you to doubt everything; it asks you to **know the scope of validity of every sentence**. Trust the experiments, because experiments can fool you; dare to question, but come back down to verification; after questioning, propose a plan - executable, falsifiable, with a control; keep conclusions

and evidence apart; admit "I don't know"; beware what "sounds self-evident"; make reproduction the first lesson; and pin everything to paper with experiment logs. - Read this chapter after finishing the project, and you will find it is not an extra burden but the foundation that lets the eleven weeks of work before it **truly hold up**.

Appendix A - Glossary

Arranged roughly in reading order. Mastering the bold-faced terms covers almost everything in this handbook.

Term	Also known as	One-line meaning
Steganography	-	Hiding a secret inside an innocent carrier so the act of communication is hidden
Steganalysis	-	Deciding whether a carrier hides a secret, from statistics or learning
cover / stego	carrier / embedded image	Original image / image after embedding
LSB	least significant bit	Lowest bit of a pixel; flipping it is visually negligible
bit plane	-	Binary layer formed by one bit position across all pixels
redundancy	-	Carrier parts that can be modified without obvious artifacts
capacity / payload	embedding capacity	Maximum secret bits an image can carry
relative payload	-	Embedded bits divided by available positions
embedding efficiency	alpha	Average secret bits per modification, alpha
matrix embedding	-	Group coding that embeds p bits while changing at most one position
Hamming code	binary Hamming code	Linear error-locating code with block $[n,k,3]$
parity-check matrix	H matrix	$p \times n$ matrix with distinct columns used to compute syndromes
syndrome	-	$s = H \cdot x \pmod{2}$; a summary of the block state
GF(2)	Galois field of two elements	Field containing only 0/1 with XOR arithmetic
shrinkage	-	Coefficient magnitude decrease lands on zero, ruining the block
wet paper coding	WPC	Embedding on dry positions while wet positions stay

Term	Also known as	One-line meaning
		untouched
dry / wet positions	-	Modifiable positions / untouchable positions
F5 / nsF5	-	JPEG matrix-embedding algorithm; nsF5 removes shrinkage with wet paper
magnitude decrease	coefficient shrink	Reducing coefficient by 1 to flip LSB parity
SHA-256	-	256-bit cryptographic hash used for image keying
keying	-	Using password/content keys to decide hiding positions
deterministic permutation	-	Shuffle where the same seed always gives the same order
splitmix64	-	64-bit PRNG used by the project
Fisher-Yates	-	Standard uniform shuffle algorithm
self-synchronization	-	Decoder finds the payload without external parameters
tamper perception	-	Detecting that an image was modified
blind steganalysis	-	Statistical detection without the original cover
chi-square test	Westfeld test	Checks whether gray-level pairs were balanced
p-value	-	Probability that observations fit the assumption; here high p is suspicious
RS analysis	-	Counts regular/singular groups under +/- masks
regular / singular	R / S groups	Groups whose discriminant rises / falls after flipping
difference entropy	-	Shannon entropy of adjacent differences; measures texture randomness
Shannon entropy	-	Information/uncertainty measure in bits
supervised learning	-	Training a model on labeled samples
feature vector	-	Numeric description of a sample (11-D v1 or 143-D v2)

Term	Also known as	One-line meaning
		here)
label	-	Ground truth (0 = clean, 1 = stego)
binary classification	-	Predicting membership of one of two classes
logistic regression	-	Linear combination + sigmoid; an interpretable classifier
sigmoid	-	S-shaped function mapping any real number to 0-1
cross-entropy loss	-	Standard classification loss
gradient descent	-	Iteratively moving parameters downhill in loss
overfitting	-	Memorizing training data at the cost of generalization
cross-validation	-	Rotating validation folds to estimate generalization
data leakage	-	Validation information entering training, inflating metrics
GroupKFold	-	Grouped cross-validation (same photo's samples stay together)
confusion matrix	-	Four-cell counts of truth vs prediction
precision / recall	-	Share of flagged that are real / share of real that are caught
ROC / AUC	-	Detection-vs-FP curve across thresholds and its area
Youden's J	-	Threshold maximizing TPR - FPR
false positive / negative	FP / FN	Clean flagged as stego / stego missed
SRM	Spatial Rich Model	High-pass residual filter family that highlights embedding noise
residual map	-	Noise residual after high-pass filtering
LightGBM / LGB	-	Gradient-boosting implementation used by the v1.4 models
feature group	-	A batch of features from one source

Term	Also known as	One-line meaning
		(BASE/SRM/PREFIX/LSB-PREFIX)
out-of-distribution	OOD	Inputs unlike the training distribution (e.g., real JPEG clean)
BOSSbase	BOSSbase 1.01	Standard benchmark: 10,000 512x512 grayscale PGM images
PGM	-	Lossless grayscale image format (BOSSbase carrier format)
stacking	meta-learner	Ensemble where a second model combines base-model predictions
memmap	memory-mapped file	Large arrays written/read in slices to avoid OOM
calibration set	-	Separate subset used only for threshold selection
grid tuning	-	Searching a hyperparameter grid and comparing models
dual-version model	-	143d robust model + 53d interpretable model deployed together
ctypes	-	Python binding mechanism for C/C++ DLLs
DLL	dynamic-link library	Windows shared library
CUDA / batching	-	GPU parallelism / processing many samples at once
consistency check	-	Bit-level comparison between C++/GPU and Python outputs

Appendix B - Command Cheat Sheet

Run from the project root F:\Steganography.

Purpose	Command
Install core dependencies	<code>pip install numpy pillow</code>
Install everything (ML/plot)	<code>pip install -e .</code>
Core algorithm self-test	<code>python src\test_core.py</code>
Steganalysis self-test	<code>python src\test_steg.py</code>
False-positive regression	<code>python src\test_false_positive.py</code>
GUI smoke test	<code>python src\test_gui.py</code>
End-to-end verification	<code>python src\run_e2e.py</code>
Launch the GUI	<code>python src\gui.py</code>
Code-family/efficiency plot	<code>python -c "import sys; sys.path.insert(0,'src'); from efficiency import plot_code_family_and_efficiency; print(plot_code_family_and_efficiency())"</code>
Generate v1 dataset	<code>python src\make_dataset.py data\campus.jpg --out campus</code>
Generate v2 dataset (12 variants / 143-D)	<code>python src\make_dataset.py --out campus_v2 --feature-set v2 --variants all</code>
Generate SRM-enhanced dataset (same source)	<code>python src\make_dataset.py data\campus.jpg --out campus_srm --preprocess srm -j 16</code>
Train/evaluate a dataset	<code>\$env:DS_FILES = "dataset.csv"; python src\train_model.py</code>
Train on v2 + JPEG clean	<code>\$env:DS_FILES = "dataset_campus_v2_jpeg.csv"; python src\train_model.py</code>
Single-image ML prediction	<code>python -c "import sys; sys.path.insert(0,'src'); from ml_predict import get_predictor; import numpy as np; from PIL import Image; print(get_predictor().predict(np.asarray(Image.open('img/cover.png')).convert('L')))"</code>
Switch to 53d interpretable model	<code>python -c "import sys; sys.path.insert(0,'src'); from ml_predict import MLPredictor; import numpy as np; from PIL import Image; print(MLPredictor(model_path='models/stego_classifier_v2_jpeg_lgb_51d.jobl</code>

Purpose	Command
	<pre> ib', clip_outliers=False).predict(np.asarray(Image.open('img/cover.png').convert('L '))))"</pre>
C++/Python consistency check	<pre>python -c "import sys; sys.path.insert(0,'src'); import cppembed; cppembed.selfcheck()"</pre>
GPU dataset generation	<pre>python gpu\make_imageset.py</pre>
GPU feature self-check	<pre>python gpu\featurize_gpu.py</pre>
GPU v2 feature self-check	<pre>python gpu\featurize_v2_gpu.py</pre>
GPU training	<pre>python gpu\train_ml_gpu.py</pre>
GPU single-image detection	<pre>python gpu\predict_gpu.py img\cover.png</pre>
BOSSbase multi-source (GPU)	<pre>python gpu\make_imageset.py data\BOSSbase_1.01 --out bossbase, then python gpu\train_ml_gpu.py</pre>
Build packages	<pre>python -m build</pre>
Install wheel	<pre>pip install dist\nsf5stego-1.9.0-py3-none-any.whl (or pip install nsf5stego)</pre>

Appendix C - 12-Week Checklist

Week	Topic	Required action	Date
1	Images & binary	Run the bit operations; state what LSB means	
2	Python environment	pip install; run run_e2e.py; save one grayscale image	
3	LSB & blind analysis	Embed -> analyze experiment; read test_steg.py output	
4	Chi-square / RS	Explain Gn collapse to a friend; run test_false_positive.py	
5	Matrix embedding / F5	Work one p=3 example; verify with matrix_demo	
6	nsF5 / wet paper	Read the wet-paper section of ns5_core.py; run its test	
7	Hash keying	Wrong-password and one-pixel experiments; write the keying boundary	
8	ML foundations	Run v1 training; explain GroupKFold and AUC	
9	ML steganalysis	Compare 11-D / 143-D / 53-D predictions; explain SRM and dual models	
10	Engineering	Run cppembed.selfcheck and featurize_v2_gpu; read SRM/multi-source pipeline	
11	Capstone	Choose a direction and finish 70% of code/experiments	
12	Presentation	Finish report and demo; answer three questions from 10.5 unaided	

Appendix D - Concept-to-Code Map

Concept	File	Read first
Message encoding / length header	src/ns5_core.py	encode_string / decode_string
Image hash / seeds	src/ns5_core.py	derive_seed / get_image_hash
Deterministic permutation	src/ns5_core.py	permute_index
Hamming code / syndrome	src/ns5_core.py	build_hamming / syndrome
Matrix embedding	src/ns5_core.py	MatrixEmbedding._embed
Wet paper solver	src/ns5_core.py	solve_wet_paper / gauss_solve_GF2
Pixel-domain nsF5	src/ns5_core.py	nsF5Pixel._embed
High-level embed/extract	src/ns5_core.py	embed_string / extract_string
Chi-square statistics	src/steganalysis.py	chi2_stats / chi2_sf
RS statistics	src/steganalysis.py	rs_metrics
Combined verdict	src/steganalysis.py	analyze
v1 feature order	src/fsfeatures.py / ml_predict.py	FEAT_KEYS / V1_FEAT_KEYS
v2 143-D features	src/featurize_v2.py	ALL_FEATURE_NAMES / featurize_v2()
SRM preprocessing	src/srm_filter.py	srm_residuals_np / preprocess_batch_torch
Training pipeline	src/train_model.py	main()
Dataset generation	src/make_dataset.py	main()
C++ embedding wrapper	src/cppembed.py	embed_string / selfcheck
Dual-model inference	src/ml_predict.py	MLPredictor(model_path=..., clip_outliers=...)
GPU v1 features	gpu/featurize_gpu.py	batch operators / check_vs_cpu
GPU v2 features	gpu/featurize_v2_gpu.py	extract_features_v2_gpu() / self-check

Concept	File	Read first
GUI callbacks	src/gui.py	button events -> core functions

Appendix E - Recommended Resources

Books

- Zhou Zhihua, Machine Learning (in Chinese) - especially Chapter 2 on model evaluation and selection;
- Hang Li, Statistical Learning Methods (in Chinese) - classic derivations of logistic regression and perceptrons;
- Gonzalez & Woods, Digital Image Processing - bit planes, histograms, and frequency-domain fundamentals (early experiments used its images; v1.4 data sources switched to campus photos and BOSSbase);
- Any Chinese textbook on information hiding and digital watermarking as broad reading.

Core Papers (in reading order)

- Westfeld, F5 - A Steganographic Algorithm, 2001 - matrix embedding and F5;
- Westfeld & Pfitzmann, Attacks on Steganographic Systems, 1999 - chi-square test;
- Fridrich, Goljan & Du, Reliable Detection of LSB Steganography in Color and Grayscale Images, 2001 - RS analysis;
- Fridrich, Goljan, Lisonek & Soukal, Writing on Wet Paper, 2005 - wet paper coding;
- Pevny, Filler & Bas, Using High-Dimensional Image Statistics to Perform Unsigned Steganalysis - modern feature-based steganalysis (SPAM) entry point.

Online

- The project README and the canonical results table docs/RESULTS.md (current numbers for the SRM / 143-D / 53-D and multi-source experiments, with corpus and protocol labels); GitHub: Yukinoshita-lin/nsf5-steganography;

- scikit-learn docs: LogisticRegression, GroupKFold, roc_curve;
- PyTorch tutorials (needed for the GPU pipeline);
- GitHub Actions documentation for CI/release.

Try it | The handbook ends here. Return to the checklist in 0.4 and grade yourself item by item. If you can tick all of them, give yourself a certificate: nsF5 Steganography - From Zero to Understanding.

Appendix F - A 6-12 Month In-Depth Self-Study Roadmap

Try it | What Introduction 0.3 gives you is the compressed **12-week "get started"** route; this appendix gives the **6-12 month "truly understand it and produce results"** route. It is for readers who want to genuinely master the three pillars - digital image processing, information hiding, and machine learning - and go on to do real research or engineering work. **Both routes use exactly the same content; only the time allocation differs.**

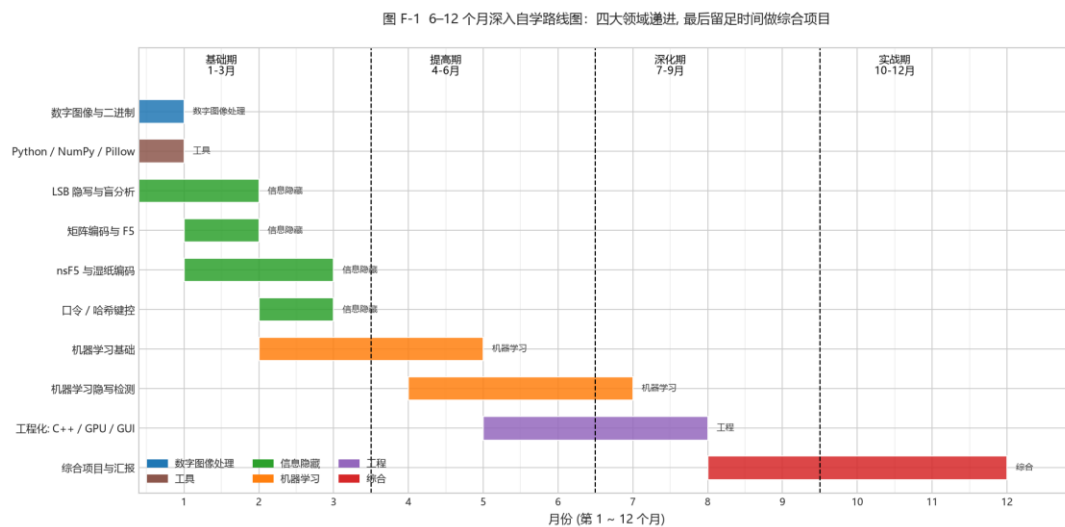


Figure F-1 (the 6-12 month Gantt chart: x-axis = month, y-axis = topic, color = field; dashed lines split the journey into four stages - Foundations (months 1-3) / Advancing (4-6) / Deepening (7-9) / Capstone (10-12))

F.1 The four stages: goals and deliverables

Stage	Months	Goal	Main content	What you can do when the stage ends
Foundations	1-3	Solidify the foundations of all three fields	ch01 digital images and binary, ch02 the Python toolchain, ch03 LSB steganography and blind analysis, ch04 matrix embedding and	Hide a sentence independently and catch it with chi-square/RS; work out a p=3 Hamming embedding by hand;

			F5, ch05 nsF5 wet-paper codes, ch06 hash keying	explain wet-paper decoding clearly
Advancing	4-6	Enter machine learning; build the feature→model→evaluation framework	ch07 machine learning foundations, ch08 ML steganalysis (11-d → 143-d → dual versions)	Explain overfitting / data leakage / GroupKFold / AUC clearly; read and reproduce the v1 11-d training pipeline
Deepening	7-9	Master the engineering and the "why behind every design choice"	ch08.8 SRM and the 143d/53d strategy, ch09 engineering (C++ / GPU / GUI / multi-source data)	Read SRM filtering and the 143-d features; understand why same-source and cross-source gains differ; run GPU batch featurization with self-checks
Capstone	10-12	Produce a complete piece of work that is genuinely "your own"	ch10 capstone project, ch11 reporting; then pick one research or engineering thread to go deep on	Complete a reproducible improvement experiment with an honest analysis of the results; defend it without slides

Tip | The "Foundations" stage is nearly identical to the 12-week route - you just spend more time each week thinking through every **why** (especially the statistical detection in ch03 and wet-paper codes in ch05). The real gap opens up in "Deepening" and "Capstone".

F.2 Four main threads: how to cover breadth, completeness, and practice

To produce results, do not spread your effort evenly. Pick one thread to dig into; for the others, "able to follow along" is enough:

Thread	Focus	Where to go deeper
Algorithm thread	nsF5/F5, matrix embedding, wet-paper codes, syndromes	ch04-05 and <code>ns5_core.py</code> in the project; outward to JPEG-domain and DCT-coefficient steganography
Detection / ML thread	Feature engineering, SRM, 143d/53d, OOD	ch08, <code>featurize_v2.py</code> , <code>train_model.py</code> ; outward to modern high-dimensional features and deep-learning steganalysis
Engineering thread	C++ DLL, GPU batching, GUI, multi-source datasets, CI	ch09, <code>cppembed.py</code> , <code>gpu/*</code> , <code>.github/workflows</code> ; outward to deployment and robustness
Research thread	Reproduce a paper → find a gap → improve → evaluate honestly	The literature (Appendix E) and <code>docs/RESULTS.md</code> ; emphasize split/calibration/test rigor (ch07)

F.3 Optional stretch topics (pick 1-2 per thread)

- **JPEG-domain steganography:** port the ch04-05 ideas to quantized DCT coefficients (the sister project `yccstego` (<https://github.com/Yukinoshita-lin/yccstego>) is exactly this direction) and feel the difference between "spatial domain vs transform domain";

- **Deep-learning steganalysis:** under small-sample constraints, how to validate a signal with interpretable features first and only then carefully introduce deep models (the lesson of ch08.1);
- **Domain adaptation / cross-source robustness:** with the v1 protocol, SRM preprocessing helped within the same source but hurt across sources (ch08.8.1; that result was not reproduced in the current version) - study how domain adaptation can make features robust across cameras/compression sources;
- **Adversarial robustness and false-positive control:** study the trade-off between "loose vs strict" thresholds and Youden / low-FP operating points (ch07), plus the physical ceiling of undetectable embedding;
- **Interpretability:** 143d is robust while 53d is interpretable (ch08.8.4) - study which features truly contribute the gain and how to explain them dimension by dimension.

Read the code | Land any of the topics above in code: reproduce the baseline first, make a small change, and use `GroupKFold` to produce an honest comparison. **"I changed A and the number went up" convinces nobody - a train/calibration/test three-layer structure with grouped splits is what proves it really went up.** (ch07 and ch08 keep hammering this point.)

F.4 How this relates to Introduction 0.3 (the 12-week route)

- **Want to get started fast:** follow 0.3, run the 12 weeks once, and work through the checklist in Appendix C;
- **Want to go deep and produce results:** follow this appendix, 6-12 months, taking 0.3's weekly tasks **slower, deeper, and better connected**, and during "Deepening/Capstone" add the parts of the 0.5 project map that only matter for research (SRM, multi-source data, GPU, dual model versions, OOD validation);

- The **shared bottom line** of both routes: never skip the hands-on experiments, and never skip honest evaluation. What this handbook most wants to teach you is not an algorithm but "**how to judge whether an experimental conclusion can be trusted**".

Think about it | Which thread will you pick, and over what time frame? (12 weeks vs 6-12 months means spending your time on "breadth" or "depth".) Write it down - it is the personal goal for your journey through this book.

Appendix H - Command-Line Tool and Installed Build (v1.9.0)

The main handbook teaches the algorithms "from the Python code outwards". Since v1.8 the project also ships two ways to complete the whole workflow **without writing a single line of code**: the graphical interface (see Ch. 2 and Ch. 10) and the command-line tool `nsf5stego`. This appendix is a quick reference for installing and using both; the commands work in Windows PowerShell and in Linux/macOS terminals alike.

Version note | This appendix is updated for **v1.9.0** (2026-10). On top of the v1.8 commands `embed` / `extract` / `analyze` / `gui`, v1.9.0 adds the **JPEG compressed domain** (`--jpeg``) and **one-command re-runs of an experiment record** (``repro``). Existing pixel-domain usage is unaffected.

H.1 Three ways to install

Form	Who it is for	How to install	Where output lands
Windows installer (recommended for end users)	No Python, just want the app	Download <code>nsf5stego-setup-<version>.exe</code> from Releases (https://github.com/Yukinoshita-lin/nsf5-steganography/releases/latest) and double-click (Simplified-Chinese wizard, per-user, no admin rights)	%APPDATA%\nsf5stego\output
Portable zip	No installation, carry it around	<code>nsf5stego-portable-<version>-win64.zip</code> from the same Release; unzip and run	The unzipped folder
Source / PyPI	Reading the code, scripting, CI	Source: <code>pip install -e .</code> ; PyPI: <code>pip install nsf5stego</code> (1.9.0 published)	The current working directory

Installer highlights:

- Start menu (desktop shortcut optional) → **nsF5 隐写工具**; double-clicking opens the GUI, and **no Python is required**;
- Ticking "add to user PATH" in the wizard makes the `nsf5stego` command available (reopen the terminal); uninstalling removes it from PATH again;
- If SmartScreen warns on first launch, choose "Run anyway" — the project is not code-signed.

Watch out | The project supports Python 3.9, but the **JPEG compressed domain depends on the companion project [`yccstego`]**(<https://github.com/Yukinoshita-lin/yccstego>), which **requires Python >= 3.10**. That dependency is installed automatically via an environment marker; when it is missing, `jpegstego.available()` is `False` and `--jpeg` reports a readable install hint, while **the pixel-domain features are completely unaffected** (the same degradation pattern as a missing `lightgbm` for ML judgment).

H.2 Subcommand quick reference

```
v = 200
nsf5stego --help                # overview; --version prints the version
nsf5stego embed  cover.png -m "secret text" -p passphrase -o stego.png
nsf5stego extract stego.png -p passphrase
nsf5stego analyze stego.png --sensitivity 宽松 --json
nsf5stego repro  experiment.json -m "secret text"
nsf5stego gui                  # GUI (identical to python src/gui.py)
```

Subcommand	What it does	Key options
embed <cover>	Embed text into an image	-m/--message (stdin if omitted), -o/-out, --jpeg, --quality (1-100, default 85, --jpeg only), --json

		(export the experiment record), --method {nsF5,matrix}, --hamming-p {1..6} (default 3), -p/--password
extract <stego>	Decode the text back	--jpeg, --method, --hamming-p, -p/--password
analyze <image...>	Blind steganalysis (chi-square + RS [+ ML])	--jpeg, -s/--sensitivity {严格,均衡,宽松} (default 均衡), --json
repro <record.json>	Re-run from an experiment record and verify item by item	-m/--message, -c/--cover, -p/--password, -o/--out
gui	Launch the GUI (tkinter)	—

Three semantics worth knowing:

- **`--method` and `--hamming-p` apply to the pixel domain only**; the JPEG compressed domain is always nsF5 and does not accept them;
- **`analyze` expands wildcards itself** (the Windows shell does not), so `nsf5stego analyze *.png` works on all three platforms; an unreadable image in a batch is recorded as an `error` item and the rest still run, with an overall non-zero exit;
- **The shape of `--json` under `analyze`**: a single image prints an object, several images print an array of objects (each carrying an `image` key), so scripts can parse it reliably.

H.3 Pixel domain vs JPEG compressed domain: the domain must match

```
v = 200
# Pixel domain (default): flips LSBs of gray pixels, writes PNG
nsf5stego embed cover.png -m "secret" -o stego.png
nsf5stego extract stego.png

# JPEG compressed domain (v1.9.0): embeds in quantized DCT coefficients, writes a
standard .jpg
nsf5stego embed cover.png --jpeg -m "secret" --quality 85
```

```
nsf5stego extract cover_stego.jpg --jpeg      # your embed used --jpeg, so decode needs it
too

# Blind analysis: with --jpeg the main signal is the |c|=1 coefficient fingerprint (ML is
unavailable there)
nsf5stego analyze stego.png
nsf5stego analyze cover_stego.jpg --jpeg
```

Watch out | In the compressed domain the payload lives in the **quantized coefficients of the JPEG bitstream**, so the stego .jpg must be stored and transferred byte-for-byte — **re-encoding it with any tool destroys the payload**. The pixel domain is the opposite: PNG/BMP are lossless, so bit planes never shift.

Decoding must match embedding in **four respects: domain (whether `--jpeg` was used), method (`--method`), Hamming parameter *p*, and passphrase**. When they disagree the output is not the original text; the CLI exits with a non-zero code and says so instead of printing replacement characters.

H.4 Experiment records and one-command re-runs (v1.9.0)

Every embed can export an **experiment record**: `embed --json` (or "export experiment record" in the GUI) prints parameters, image hashes and modification statistics as JSON with schema `nsf5stego.experiment/1`. The record **contains no plaintext**, so a re-run needs the original text again.

```
v = 200
nsf5stego embed cover.png -m "secret" --json > experiment_20261003.json
nsf5stego repro experiment_20261003.json -m "secret" # re-run and verify item by item
```

The verification strength of `repro` depends on the domain:

- the **pixel domain** is always **byte-deterministic**;

- the **JPEG domain** is byte-deterministic too when the `yccstego` version matches, and automatically falls back to **round-trip verification** (extraction must agree) when the version recorded in the record differs — older records stay compatible.

The passphrase is recorded **only as present/absent, never its content**, so `repro` needs you to pass it again (`-p`).

H.5 Exit codes and common errors

Exit code	Meaning
0	Success
1	Runtime failure: capacity exceeded / passphrase or parameters mismatched / unreadable image
2	Argument error (argument-parsing layer)

Scripts can judge success straight from the exit code. The three mistakes newcomers hit most:

1. **Decoding with `--jpeg` after embedding without it** (or the reverse) — the domains disagree, so nothing decodes;
2. **A mismatched `p` or passphrase** — the output is invalid text; the CLI exits non-zero with a hint;
3. **Using `--jpeg` on Python 3.9** — `yccstego` is missing; the error carries an install hint, and installing it or switching to the pixel domain fixes it.

H.6 The GUI: the whole loop in one minute

The GUI loads a demo cover on first launch, and a guidance bar at the top tracks your progress and always tells you where to click next: press **"Embed and save"** (watch the

modified-pixel count and the output path) → tick "**Show diff**" to zoom in on which pixels changed → press "**Decode / extract**" to confirm the text comes back intact. The whole loop takes under a minute; only then go back to the matching chapter to understand why it works.

The GUI and the CLI share one set of parameter semantics (domain / method / p / passphrase); the CLI entry point is `src/cli.py`, while the algorithm core is still `ns5_core.py`.

To go further: the command line only runs the workflow — **the principles behind the options** are still in the main text. Domains and bit planes are in Ch. 1, JPEG and DCT in Ch. 1½, matrix embedding and wet paper in Ch. 4 and 5, and the design of experiment records in Ch. 9.

Appendix J - Exercise Hints and Answers

This appendix gives only "hints + key numbers"; work out the full reasoning yourself first — an exercise only counts once you have checked the answer. The problem numbers correspond to the end-of-chapter exercises of each chapter.

J.1 Chapter 1

P1.1: $156 = 10011100_2$, $\text{LSB} = 0$. Clear-then-set: $156 \& 0xFE = 156$, then $| 1 \rightarrow 157$; XOR: $156 \wedge 1 = 157$. Both methods give the same result.

P1.2: After embedding [133, 132, 131, 129, 134, 130, 129, 135], with changes [+1, -1, +1, 0, 0, -1, +1, 0] (bit-for-bit identical to the table in Section 3.2).

P1.3: After random bits are embedded into an all-zero image, every pixel is rewritten somewhere between 0/1 — the histogram shifts from "concentrated at 0" to "half 0 and half 1", and the value pair (0,1) is flattened completely; visually it is still all black (value 1 $\approx$ black), but the chi-square statistic shoots up instantly. The visual redundancy is still there; the statistical redundancy is gone.

P1.4: The MSB plane is heavily skewed (for positive integers the MSB = 1 share is high), and the lower a bit plane is, the closer to 50% it gets — the LSB plane is the one that "looks most like a coin toss".

J.2 Chapter 1½

P1½.1: Quantized DC = $\text{round}(-224 / 5) = -45$; dequantized, $-45 \times 5 = -225$; reconstructed mean brightness = $-225 \div 8 + 128 = 99.875$, off from the original 100 by 0.125 — that is the concrete size of "lossy".

P1½.2: $+3 \rightarrow +2$ is a decrement (magnitude -1, sign kept, LSB flipped); $+3 \rightarrow +4$ also flips the LSB, but it is an **increment** — it pushes energy toward high frequencies, violates

the nsF5 discipline of "minimal modification", and leaves a detectable "high-frequency boost" signature on the coefficient histogram. The first principle of steganographic modification: if the goal can be reached with the smallest, direction-certain change, do not introduce new statistical traces.

P1½.3: Higher quality → smaller quantization steps → fewer coefficients rounded to zero → more nonzero AC coefficients (carriers) → more capacity. Conversely, a low-quality JPEG is itself a "big cleanup".

P1½.4: Pixel-domain capacity $\approx$ 8 KB with 108 pixels modified; JPEG-domain capacity 205 bits with 117 coefficients modified — compare the table in Section 1.5.3.

J.3 Chapter 2

P2.1: In uint8, $250 + 10$ wraps around to 4; $3 - 5$ underflows to 254 (modulo 256).

P2.2: `b = a.copy()` or `b = np.array(a)`; for a copy, `b.base` is `None`, while for a view, `b.base` is `a`.

P2.3: Bits = bytes $\times$ 8, so $126 \times 8 = 1008$ (exactly twice 504, because the length header is only 16 bits and does not double along with the message); the modification count does not double — it is decided randomly by the "hit rate" between message bits and pixel LSBs: the expectation doubles, while the actual count fluctuates.

P2.4: `img[:, ::2]` takes every other column horizontally, making the picture thinner — but it is still a view of the original array, and modifying it changes the original.

J.4 Chapter 3

P3.1: The pair sums stay [16, 12, 18, 14], expectations [8, 6, 9, 7], deviations [+2, 0, 0, 0], $\chi^2 = 4/8 = 0.5$, `df = 3`, `chi2_sf(0.5, 3)  $\approx$  0.919` — a very high p-value, with three of the four pairs completely flattened: a verdict of "suspected randomization".

P3.2: [128,128,131,131]: original $f = 3$; F flip $\rightarrow$ [127,127,132,132], $f = 5$; M flip $\rightarrow$ [129,129,130,130], $f = 1$. [200,201,202,203]: original $f = 3$; F $\rightarrow$ [199,202,201,204], $f = 7$; M $\rightarrow$ [201,200,203,202], $f = 5$. The first group is smoother under M (contributing to the R_m side); the second group gets rougher under both flips.

P3.3: Before embedding, all counts pile up at gray level 0, so the (0,1) pair gives a huge χ^2 and a p-value ≈ 0 (verdict "clean"); after random bits are embedded, the (0,1) pair flattens instantly and $p \approx 1$ (verdict "suspicious") — the more unbalanced the background, the stronger the chi-square signal after embedding. Blind spot: embedding all-zero bits does not change a single pixel — completely traceless.

P3.4: Full-capacity embedding rewrites nearly all LSBs, the even-odd pairing is completely flattened and the bit plane completely randomized — both prob and G_n collapse to the extremes. "The more you hide, the safer" holds visually and is exactly the opposite statistically.

P3.5: The chi-square pair flattening still happens but concentrates in the mid-brightness range; the RS structural collapse weakens because "textured regions are avoided" — this is exactly the design motivation of adaptive steganography (the HUGO/S-UNIWARD family), and the SRM features of Section 8.8 were aimed straight at them.

J.5 Chapter 4

P4.1: $s = H_1 \oplus H_2 \oplus H_4 \oplus H_7 = [0,0,0]$; $d = s \oplus m = [0,1,1] \rightarrow H_6 \rightarrow$ flip bit 6, codeword [1,1,0,1,0,1,1]. Error-correction drill with bit 5 wrong: $s(y) = [0,1,1] \oplus H_5 = [1,1,0]$; if the target m were known, $s(y) \oplus m = [1,1,0] = H_3$, locating bit 3. Note that "error correction requires knowing m or assuming s should be 0" — the stego extraction side does not know m and therefore performs no error correction; this is exactly the difference in goals between steganography and channel coding.

P4.2: For $p=2$ the H columns are $[1,0]$, $[0,1]$, $[1,1]$, and $\alpha = 2 \times 4/3 = 8/3$. For 600 bits: $p=2$ needs $300 \text{ blocks} \times 3 \text{ bits} = 900 \text{ positions}$, modifying $300 \times 3/4 = 225 \text{ bits}$ on average; $p=3$ needs $200 \text{ blocks} \times 7 = 1400 \text{ positions}$, modifying $200 \times 7/8 = 175 \text{ bits}$ on average. $p=3$ spends more positions to buy fewer modifications — always a good deal when capacity is to spare.

P4.3: The premise is that "cover LSBs are approximately random". With an all-zero message, m is always 0, the no-change condition becomes $s = 0$, and the probability is still $1/2^p$ — the real warning is that a low-entropy message can be exploited by a "known-message attack" (an attacker who guesses the message can work backward to the embedding positions), which is why the keying of Chapter 6 matters.

P4.4: The modification count approximately doubles — the syndrome is decided jointly by "cover LSBs and the message"; when the message doubles, the block count doubles while the per-block modification probability stays the same.

P4.5: For 16 384 bits in a 512×512 image (262 144 positions), $p=3$ needs 5 462 blocks and $p=6$ needs 2 731 blocks — capacity suffices either way. Pick a large p to minimize modifications, a small p for localization granularity and robustness — the project default $p=3$ is the compromise point.

J.6 Chapter 5

P5.1: For $m=[0,1]$, $d = [0,1] \oplus [0,1] = [0,0]$; the zero vector is not among H 's columns $\rightarrow$ no modification needed. For $m=[1,1]$, $d = [1,0]$, and among the dry positions both position 1 and position 4 have a column that hits — the solver takes the first one in a deterministic order (the same input always yields the same plan).

P5.2: $xv = [+3, 0, +1, +6, +2, +5]$; wet points = position 2 (0) and position 3 (+1); flipping position 4: $134 \rightarrow 133$ ($+6 \rightarrow +5$, a legal decrement).

P5.3: The wet/dry split constrains "where changes are allowed" **only on the embedding side**; the extraction side simply reads the LSBs of all carriers in the same order (after a dry point is flipped, $|c|$ is still ≥ 1 , so it never disappears from the carrier set). If the wet points were instead decided by a passphrase-driven pseudorandom sequence, the embedding would modify positions outside the wet set it itself drew — the extraction side could no longer know "which bits are trustworthy", and self-synchronization would fail. The beauty of content derivation is exactly this: the embedding touches only dry points; after dry flips the image content has changed, but "which positions were flipped" is verified by the extraction side from the message itself — no need to recompute the wet points.

P5.4: On smooth images (such as the gradient demo) a large share of pixels fall on 127/128/129, so the wet-point density is high — nsF5 steers around them while matrix embedding pays no attention; the latter keeps hitting mines and retrying, and its modification count ends up higher. Conclusion: algorithm comparisons must fix the test image — "which is better" is relative to the image statistics.

P5.5: When every carrier is a wet point, the dry set is empty and the equations generally have no solution — the project raises `CapacityError` (compare the 16×16 all-128 case of `test_capacity_error`), hinting to switch to a larger image / higher quality / shorter message.

J.7 Chapter 6

P6.1: After the first state advance, $x = 9e3779b97f4a7c15$ (the golden constant itself), and the output is 16 294 208 416 658 607 535.

P6.2: There are three sticking points — ① the header pool needs `seed0` (derived from the passphrase), and with a wrong passphrase the "hash" read out is garbage; ② the `seedB` derived from the garbage hash shuffles the body pool into a completely different

order; ③ even if the bits happen to read back, the length header and the content-hash check both fail. Given a second image with the same passphrase, an attacker can compare whether the two images' header-pool positions coincide (the same seed0) — this is why "multiple images, one passphrase" requires rotating the master key.

P6.3: In a teaching project the attacker model is "an ordinary opponent who has the images and the source code", and a content hash is enough to make a wrong passphrase and image tampering fail reliably; a real forensic product faces strong attackers who can re-embed an entire image, and must use a keyed MAC to restrict "who can compute a valid hash" to key holders.

P6.4: It would succeed — re-saving as PNG leaves the pixels unchanged, so the LSBs and header bits are fully intact. The reason hash awareness is not triggered: in the pixel domain the anchor is the byte hash of the **original cover**, with the semantics "repro/experiment archives compare against the original image", not "compare against the stego image"; the JPEG domain has a more ingenious design — the anchor is the chroma hash, the embedding touches only luma Y, and as long as the chroma is unchanged the check passes. Understand the layers, and each sits where it belongs.

P6.5: Measured, about 47%–53% of bit flips (e.g., 121/256 in Section 6.4) — averaged over many runs it lands very close to 128/256.

J.8 Chapter 7

P7.1: $\eta=10$: $w = 0 - 10 \times (-0.1) = 1.0$, new loss about 0.349 (still falling); $\eta=100$: $w = 10$, the clean sample's prediction is pushed close to 1, and the loss rises to about 1.54 — the step is too large, and one stride jumps across the valley floor. The larger the feature scale, the more dangerous the same learning rate becomes.

P7.2: Grouping by the pseudo photo_id (same-source samples in the same fold) is the sounder choice — otherwise "sibling samples" appear on both the training and validation sides, inflating the validation score.

P7.3: The false positive (FP) — judging a clean image as stego. In a forensic setting that means casting suspicion on the innocent, a social cost higher than missing one weakly embedded image.

P7.4: sklearn's LogisticRegression applies regularization and a different optimizer by default; the direction should match the hand calculation (w positive) while the numbers need not — exactly the chance to feel the difference of "one model, different solvers".

P7.5: The mistake is comparing scores across different corpora (RESULTS.md: numbers from different corpora are not comparable) — 0.89 vs 0.81 reflects corpus difficulty, not model strength.

J.9 Chapter 8

P8.1: Ranked by how much domain knowledge is built in: yenet (fixed SRM kernel front-end) > 143d (SRM statistical features) > 53d/11d (handcrafted statistics) > xuNet (no domain knowledge). The AUC ranking agrees roughly ($0.9541 > 0.8062 > 0.7172 \approx 0.7128 > 0.5007$) — but 143d trailing yenet reminds us that beyond "putting knowledge in the right place", the expressive power of the end-to-end architecture is also a variable; the two are not two knobs on the same machine.

P8.2: A perfect model's ROC is the "bent line hugging the top-left corner": a 0% false-positive rate at 100% detection, $AUC = 1.0$; a random model's ROC is the diagonal, $AUC = 0.5$.

P8.3: Sources of the difference: your photo style (content complexity), the shooting device (noise floor), whether the grouping is strict, the sample size — any one of them is enough to push the AUC away from the README.

P8.4: The symmetric problem appears — the model takes PNG's characteristic "uncompressed noise" as an anomaly, creating new false-positive/false-negative shifts for a real JPEG deployment scenario. Aligning the training distribution with the deployment distribution is a higher priority than tuning hyperparameters.

P8.5: 53d is the mandatory pick for explaining the decision basis to non-technical audiences/reviewers and for compliance scenarios that need to audit what each dimension means; 143d is the mandatory pick for bulk screening in online services (accuracy first) and as a research baseline against strong embedding variants.

J.10 Chapter 9

P9.1: The expected top three are the deterministic permutation, the inner loops of the feature statistics, and file IO — the first two are the targets Section 9.2 sinks down, and the third can be optimized with buffered reading and writing.

P9.2: Fix the seeds and inputs → call the two versions of the permutation → diff bit by bit (`np.array_equal`) → repeat over several seeds and sizes, and pass only when everything agrees.

P9.3: This project's positioning includes "usable without acceleration" (servers / CI / teaching machines), and the fallback guarantees identical functionality and results; if performance were a hard requirement (say, an online service), the degradation should instead be "a clear error + require installation", avoiding a silent slowdown.

P9.4: Changing algorithm parameters first trips the unit layer (the round-trip assertions in `test_core`); a file missing from the package first trips the system layer (items 4/5 of the frozen-build smoke test).

J.11 Chapter 10

P10.1: An example of a good point of suspicion — "Of the gain of 143d over 11d, how much comes from SRM and how much from the data expansion?" Reproduction steps: fix the photo grouping and train 11d and 143d separately; acceptance criterion: under the same protocol, an AUC gap ≥ 0.02 with non-overlapping confidence intervals.

P10.2: The hypothesis must be falsifiable ("the new feature improves AUC by ≥ 0.02 "), the protocol must group by source image, and the acceptance criterion goes into the experiment script.

P10.3: When replaying the recording, listen above all for: whether the corpus and protocol behind the numbers were stated clearly, and whether "correlation" was passed off as "causation".

J.12 Chapter 11

P11.1: The key to judging the statements is citing technical facts — for example, the point of ② is "steganography's concealment was used to deceive a specific audience", not "steganography is a bad technology".

P11.2: An example upgrade plan (hash keying $\rightarrow$ HMAC): the passphrase derives an HMAC key via PBKDF2; all key fields of the header and body switch to HMAC coverage; acceptance = all existing round-trip tests green + new "tampered fields must fail" cases.

P11.3: Example locations — the α formula in the README and the implementation in `efficiency.py`; the 0.8062 in RESULTS.md and the evaluation output of `train_model.py`; the GUI's 0.2% hint and the statistics report of `_embed`.

Appendix K - Measured Data Quick Reference (For Cross-Checking Experiments)

This appendix collects the real numbers cited throughout the handbook, all of them reproducible from the project's scripts (sources noted in parentheses). Authoritative sources: docs/RESULTS.md (generated by experiments/build_results_table.py — do not edit it by hand) and the measured output of each script.

K.1 Embedding measurements (256×256 demo image, 2026-10-04)

Metric	Pixel domain (p=3)	JPEG domain (p=3, q85)
Total carriers	65 536 pixels	5 150 nonzero AC coefficients
Modifications to embed 240 bit	108 pixels (0.2%) (240 bit = 28-byte message + 16-bit length header)	117 coefficients
Theoretical capacity	≈ 3.4 KB of text (p=3)	2 205 bit
Fidelity	No visible difference; PSNR ≈ 76 dB (measured)	No visible difference (JPEG compression noise floor)

K.2 Full-capacity efficiency sweep (random noise image, measured by compute_comparison(6))

p	Theoretical α	Measured α	Modified pixels
1	2.00	2.01	32 502
2	2.67	2.62	16 655
3	3.43	3.38	8 263

4	4.27	4.18	4 149
5	5.16	5.08	2 054
6	6.10	5.90	1 035
504 bit (p=3)	3.43	2.68	188

A note on the counting basis (do not mix these up). The first six rows of the table above are measurements of **full-capacity filling** (every available position for each p is filled), not the result of one fixed message; the last row separately lists the measurement for the 63-byte (504 bit) demo message from run_e2e.py: same image, p=3, modifying only 188 pixels. The two numbers have different denominators — the "188 pixels modified" in the Ch. 2 run_e2e output is the latter, and the 8 263 in the Ch. 4 efficiency section is the former.

K.3 Main detection table (BOSSbase 1.01, held out by source image, excerpted from docs/RESULTS.md)

Model	Features	AUC	95% CI
LGB-11d	11-D	0.7128	[0.6913, 0.7360]
LGB-143d	143-D	0.8062	[0.7894, 0.8225]
LGB-53d	53-D	0.7172	[0.6943, 0.7406]
xuNet	Raw pixels	0.5007	[0.5004, 0.5012]
ynet	30 fixed SRM kernels + conv	0.9541	[0.9463, 0.9630]

K.4 Datasets and protocol essentials

Dataset	Size	In one sentence
bossbase_npz	2 000 source images × 5 variants = 10 000 samples	The field's standard benchmark
campus_v2_jpeg	414 campus photos × 14 = 5 796 samples	Self-built dataset containing real JPEG clean images
ood_real_jpeg	1 514 real clean photos	False-positive evaluation before deployment (campus+DIV2K+ALASKA#2)

Protocol red lines: all training/evaluation is **grouped by source image**; confidence intervals are resampled by source image; numbers from different datasets are not comparable. Historically, violating these three rules produced an inflated-AUC incident of 0.8946 → 0.7555 (audit of 2026-09-14; see Section 1 of RESULTS.md for details).

K.5 Tampering and hash measurements

Item	Measured value
SHA-256 avalanche (1-character difference)	121 of 256 bits flipped ≈ 47%
JPEG-domain modified coefficients / carriers	117 / 5 150 (240 bit, p=3, q85)
JPEG-domain chroma hash anchor	Covers only Cb/Cr; never touched by the Y-channel embedding
Freeze smoke test	Six items: size/version/round-trip/ML bit-for-bit/JPEG round-trip+repro/GUI theme

K.6 LSB embedding-rate sweep (Ch. 3, ch03_results.csv)

Per-level data for the sweep experiment in Section 3.6 of Ch. 3 (256×256 demo image, random bits, 2026-10-04). The four fingerprint columns move in the same direction: the more you hide, the lower the chi-square statistic, the smaller the RS gap G_n , and the closer the LSB difference entropy gets to 1 — visually there is still no difference, yet statistically the image moves step by step toward "suspicious". For the complete 15 columns, see figures/ch03_results.csv in StegoLab.

Embedding rate	Payload bits	Modified pixels	PSNR (dB)	Chi-square statistic	RS_ G_n	LSB difference entropy
0.0%	0	0	—	34 829	0.555	0.785
10.0%	6 554	3 272	61.2	30 535	0.515	0.812
20.0%	13 107	6 488	58.2	25 995	0.471	0.842
30.0%	19 661	9 834	56.4	21 272	0.425	0.872
40.0%	26 214	13 131	55.1	16 668	0.375	0.901
50.0%	32 768	16 451	54.1	12 228	0.320	0.928
60.0%	39 322	19 719	53.4	8 398	0.261	0.951
70.0%	45 875	23 056	52.7	5 018	0.198	0.971
80.0%	52 429	26 326	52.1	2 353	0.134	0.986
90.0%	58 982	29 590	51.6	679	0.070	0.996
100.0%	65 536	32 911	51.1	72	-0.004	1.000

Appendix L - User Guide for the Companion Tool

ImageMatrix

This appendix is the complete guide to ImageMatrix, the companion tool of Chapter 1. It is an independent open-source project released by the same author (github.com/Yukinoshita-lin/ImageMatrix, Apache-2.0 license), and **has no dependency on the nsF5 project studied in this handbook, nor any steganography functionality**; its only purpose is to turn "an image = a matrix of numbers" from an abstraction into something you can see and modify on your own screen.

L.1 What It Is and Why It Comes with the Handbook

In one sentence: a **bidirectional converter between pictures and numeric matrices (R/G/B/grayscale)** — written in pure C/C++, depending on no third-party libraries, with a built-in Win32 graphical interface and a command-line mode as well.

Capability	Description
Open images	BMP / PNG / JPEG / GIF / TIFF; files can be dragged and dropped straight into the window
Export TXT matrix	Five layouts: CHANNELS / RGB / RGBA / GRAY / HEX
Import TXT matrix	Rebuild an image from edited matrix text; when width/height are missing, a dialog asks for the width and auto-corrects
Save as image	PNG / JPEG / BMP / GIF / TIFF
Pixel values	The status bar shows in real time the R/G/B/grayscale values and #RRGGBB of the pixel under the mouse

It sits in Chapter 1 because it fills precisely the only "hands-on gap" at the start of this handbook: the original project tells you to "open the project and read the code", but reading the code requires knowing Python first (not taught until Chapter 2); ImageMatrix lets you verify with your own hands that "an image is a table of integers" **before learning any code**. It is the first teaching aid in this handbook for the CSAPP discipline of "get your hands on the real thing first".

L.2 Getting and Installing It

Method	How	Who it is for
One-click installer	Download ImageMatrix_Setup.exe from the repository's Releases and double-click to install (no admin rights needed)	Anyone who just wants to run the experiments hands-on
Build from source	Run build.bat with MinGW-w64 to get build\imagematrix.exe and imagematrix-cli.exe	Anyone who wants to read a pure C++ project along the way
Run directly	The build output is a statically linked single file; copy it to another Windows machine and double-click to run	Lab machine rooms, environments without network access

The installer unpacks the program into %LOCALAPPDATA%\Programs\ImageMatrix and automatically creates Start menu and desktop shortcuts; you can uninstall it in one click under "Settings → Apps".

L.3 The Interface and Core Operations

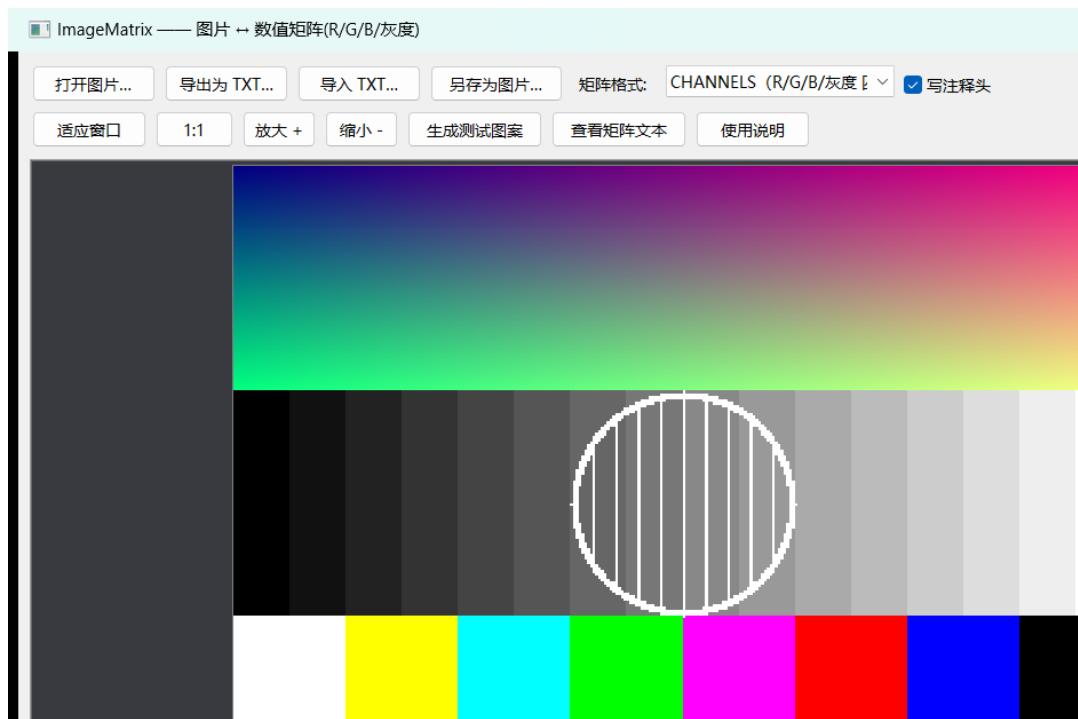


Figure L-1 The main interface of ImageMatrix. The top row holds "Open image / Export to TXT / Import TXT / Save as image", plus the "Matrix format" dropdown and the "Write comment header" toggle; the central canvas supports wheel zoom and drag panning. With no material on hand the first time, click "Generate test pattern" to get a test image with color bars + grayscale + a gradient. (Screenshot from the ImageMatrix repository)

1. Open an image: click "Open image...", or drag an image straight into the window;
2. View the matrix: click "View matrix text" and the window shows a matrix text preview of the current image's numeric matrix, which you can copy to the clipboard;
3. Export: pick a layout in "Matrix format", click "Export to TXT...", and choose where to save;
4. Import: click "Import TXT..." and the program rebuilds the image from the matrix (when the # width header is missing, a dialog asks for the width);
5. Save: click "Save as image..." to store the rebuilt result as PNG / JPEG / BMP and so on.

Keyboard shortcuts: Ctrl+O to open, Ctrl+S to export TXT, Ctrl+T to import TXT, Ctrl+E to save as an image; the wheel zooms centered on the cursor, left-button drag pans, and at magnification above 8x the pixel grid is displayed.

L.4 The Five Matrix Formats at a Glance

format	Content of each line	Typical use
GRAY	One line = one row of the image (width grayscale values)	The LSB experiments of Sections 1.2 / 1.4 (only brightness matters)
CHANNELS	R matrix → G matrix → B matrix → GRAY matrix	Seeing clearly that "a color image = three tables stacked together"
RGB	One line = one pixel: R, G, B	Looking at color pixel by pixel
RGBA	One line = one pixel: R, G, B, A	Images with transparency (A)
HEX	One line = one pixel: RRGGBB	Getting a feel for hexadecimal notation (Section 1.2)

Exported files carry a # comment header declaring width / height / format / max and the grayscale formula, and it is what makes the file come back exactly on import; the grayscale formula matches the handbook: $Y = \text{round}(0.299*R + 0.587*G + 0.114*B)$ (ITU-R BT.601).

L.5 Command Line (Batch Processing and Self-Test)

```
v = 200
build\imagematrix-cli.exe to-txt -i photo.png -o photo.txt -f GRAY
build\imagematrix-cli.exe to-image -i photo.txt -o back.png
build\imagematrix-cli.exe info -i photo.png
build\imagematrix-cli.exe gen -o pattern.png -w 320 -h 240
build\imagematrix-cli.exe selftest :: 20 项自检
```

Options: -f RGB|RGBA|GRAY|CHANNELS|HEX, --no-header, --width N, -q 92 (JPEG quality).

Repository tests verify that the TXT exported by the command line and by the GUI has **exactly identical SHA256 values** — this is the "consistency first" engineering discipline of Chapter 9, just applied to a much smaller program.

L.6 How It Maps to the Handbook Chapters

Handbook location	What to do with ImageMatrix
1.1 Images as matrices	Export GRAY / CHANNELS and see with your own eyes that "an image = a table of numbers"
1.2 Binary and the LSB	Change 200 into 201 in the TXT and experience an LSB flip with zero code
1.4 Bit planes	Write a column of pixels out in binary and observe the regular difference between high and low bits
1.5 cover / stego	Craft your first cover / stego pair by hand in Notepad
2.2 Thinking about images with NumPy	Use it to see the phenomenon first, then reproduce the same array with NumPy
3.2 Minimal LSB steganography	Replace the manual steps with code and compare "manual vs code"
9.5 Testing and CI	Refer to its selftest and GUI smoke test (GUI and CLI output are byte-for-byte identical)

L.7 A Complete Hands-On Walkthrough

Try it | From a photo to a pair of cover / stego (zero code the whole way)

- ① Open a real photo (or click "Generate test pattern");
- ② Choose the GRAY format and export it as cover.txt, then save another copy as stego.txt;
- ③ Open stego.txt in Notepad and, at the agreed positions, change a few pixels to the target parity (hiding 1 → odd, hiding 0 → even);
- ④ Import stego.txt and save it as stego.png;
- ⑤ Put the original and stego.png side by side: indistinguishable to the naked eye.

Going further: run the Chapter 3 Python LSB script, export the output/stego_*.png it produces, then open that in ImageMatrix — you will see that what the code does is exactly what you did by hand.

One final word on the boundary: ImageMatrix helps you **see** the numbers, not **hide** them. Real nsF5 steganography — Hamming matrix encoding, magnitude reduction, wet paper coding, passphrase keying — still lives in Chapters 4, 5 and 6, to be learned step by step along the code in F:\Steganography. ImageMatrix is only the stepping stone that walks you to the door.